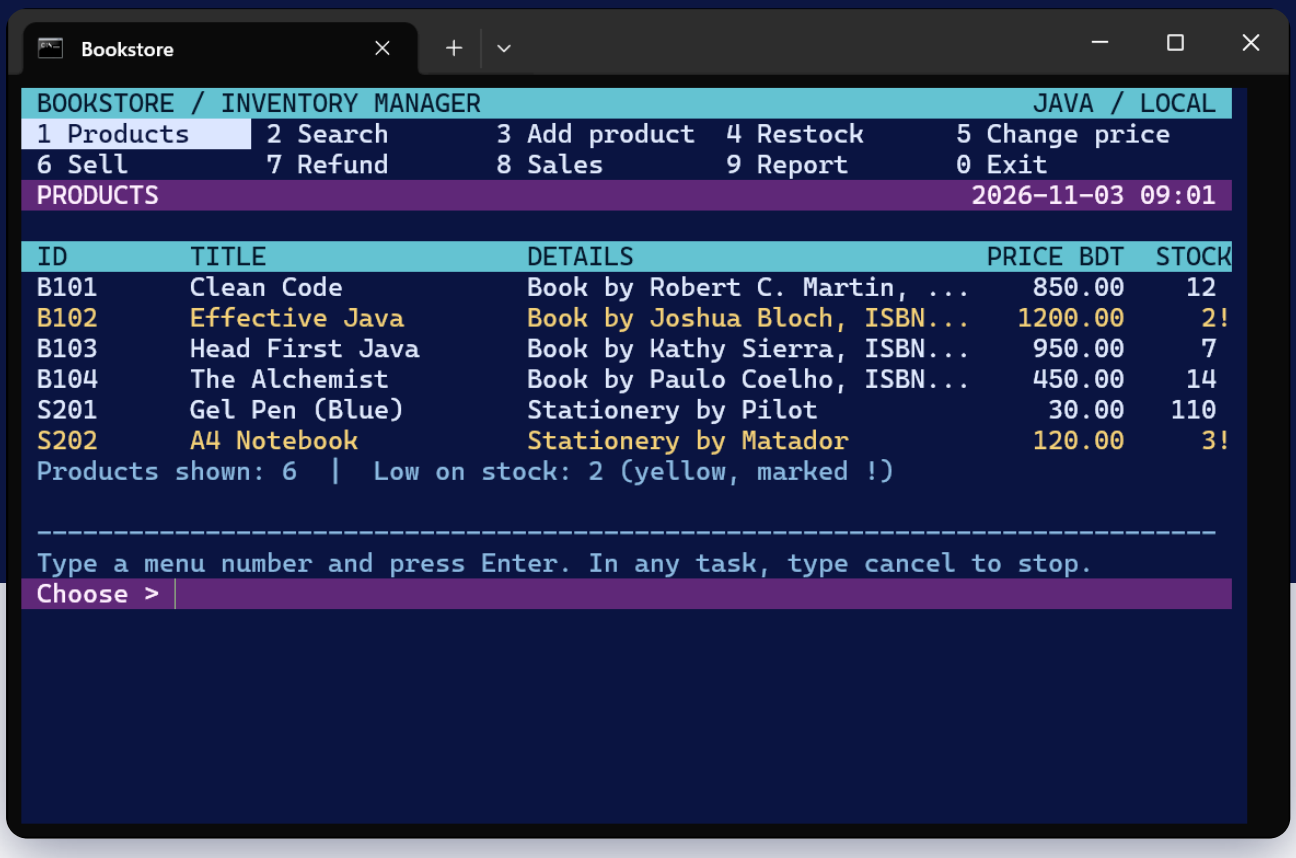
OBJECT-ORIENTED PROGRAMMING · PROJECT REPORT

Bookstore Inventory Manager

A Java console application with a full-color terminal interface



LANGUAGE
Java 17

CODE
4 files · 617 lines

DATE
November 2026

Source code: https://github.com/AtlasRoX/Bookstore-JAVA

Contents

1 Introduction 3
1.1 Objectives 3
1.2 Project at a glance 3
2 Features 4
3 Tools and Technologies 5
4 How to Compile and Run 6
4.1 Easiest way: run.bat 6
4.2 By hand 6
5 System Design 7
5.1 File structure 7
5.2 How the parts work together 7
5.3 Class diagram 7
5.4 Program flow 9
6 OOP Concepts in the Code 11
6.1 Abstraction 11
6.2 Encapsulation 11
6.3 Inheritance 12
6.4 Polymorphism 13
6.5 Static vs Non-Static 13
6.6 Method Overloading 14
6.7 Method Overriding 15
6.8 The this Keyword 16
6.9 The super Keyword 16
6.10 Subclass Reference vs Superclass Reference 17
6.11 Diamond Problem Resolution 18
6.12 Other Java features used 19
7 User Interface and Screens 21
7.1 Screen layout 21
7.2 How the colors are drawn 22
7.3 Products and Search 23
7.4 Add product 24
7.5 Restock and Change price 25
7.6 Sell 27
7.7 Refund 29
7.8 Sales history and Report 31
7.9 Cancelling a task 32
8 Input Validation and Error Handling 33

8.1 Ask-again questions 34
8.2 Rules inside Bookstore 35
9 Data Storage 36
10 Business Rules 37
11 Testing 38
12 Limitations and Future Work 40
13 Conclusion 41
A Appendix: Full Source Code 42
A.1 Product.java 42
A.2 Bookstore.java 45
A.3 BookstoreUI.java 48
A.4 Main.java 53
A.5 run.bat 56

1 Introduction

The **Bookstore Inventory Manager** is a Java console application for a small bookstore that sells books and stationery. Staff use it to keep the product catalogue up to date, receive stock, sell items for cash (with an optional member discount), refund sales and check simple business reports. All data is saved automatically to a file, so nothing is lost when the program is closed.

The project was built for the **Object-Oriented Programming** course. Its second goal is to show, inside a real working program, every OOP concept covered in the course: the four pillars (abstraction, encapsulation, inheritance and polymorphism), static and non-static members, method overloading and overriding, the `this` and `super` keywords, subclass and superclass references, and the interface diamond problem.

1.1 Objectives

- Manage a catalogue of two product types (books and stationery) that share common data but differ in details.
- Handle the daily work of a shop: adding products, restocking, changing prices, selling, refunding and reporting.
- Check every input and explain mistakes clearly, so the program never crashes and wrong data is never saved.
- Keep the code short and easy to read, split into four files with one clear job each.
- Use and clearly demonstrate all OOP concepts from the course, each one inside a real feature.

1.2 Project at a glance

Item	Details
Language	Java 17 (standard library only, no external libraries)
Program type	Console (terminal) application with a full-color screen layout, 80 × 24 characters
Source files	4 Java files, 617 lines in total, plus `run.bat`
Classes	7 classes (Product, Book, Stationery, Bookstore, Sale, BookstoreUI, Main) and 3 interfaces (Item, Trackable, Promotable)
Data storage	Java object serialization to `bookstore.dat`
Testing	30 automated test cases (30 passed) and 50 random-input sessions (0 failures)
Source code	https://github.com/AtlasRoX/Bookstore-JAVA

2 Features

The program opens on the Products screen. A menu of numbered tabs is always visible at the top; the user types a number and presses Enter.

Key	Screen	What it does
1	Products	Shows every product in a table: ID, title, details, price and stock. Low stock (3 or fewer) is yellow and marked “!”.
2	Search	Finds products by any part of the ID, title, author, brand or ISBN (not case-sensitive).
3	Add product	Adds a book (title, price, stock, author, ISBN or “-” for none) or a stationery item (brand).
4	Restock	Adds received units to a product’s stock.
5	Change price	Sets a new unit price. Old receipts keep the price that was paid.
6	Sell	Sells one product: quantity, optional 5% member discount, cash received, change, and a printed receipt.
7	Refund	Refunds a whole receipt and puts the units back in stock. A receipt can be refunded only once.
8	Sales	Lists every receipt with date, product, quantity, total, change and status (refunded ones in yellow).
9	Report	Income after refunds, units sold, number of products, low-stock count, units in stock, products added this session and stock value.
0	Exit	Closes the program. Every change was already saved when it was made.

Help while typing. Every question is shown in a purple input bar. A wrong answer produces a red message that explains the problem, and the same question is asked again (earlier answers are kept). Typing `cancel` at any question stops the current task without changing anything.

3 Tools and Technologies

Tool / technology	Used for
Java 17 (JDK, `javac` and `java`)	Writing, compiling and running the program
Java standard library	`ArrayList`, `Scanner`, `String.format`, `LocalDate` / `LocalDateTime`, object streams
ANSI escape codes	Colors, clearing the screen and the window title in the terminal
Windows Terminal	Showing the program in a window sized exactly 80 × 24 characters (opened by `run.bat`)
Java object serialization	Saving and loading all products and sales in one step
Git and GitHub	Version control and publishing the source code

4 How to Compile and Run

Requirement: JDK 17 or newer, with `java` and `javac` on the PATH.

4.1 Easiest way: run.bat

Double-click `run.bat` (or type `.\run` in a terminal in the project folder). It compiles all files and opens the program in its own Windows Terminal window that is exactly the size of the program (80 × 24). If Windows Terminal is not installed, a classic console window of the same size is used instead. Choosing **0 Exit** closes the window. If something goes wrong (for example a damaged data file), the window stays open so the message can be read.

run.bat lines 1–10

```
@echo off
rem Compiles the program, then opens it in its own window that is exactly its size: 80 x 24 characters.
cd /d "%~dp0"
javac *.java || (pause & exit /b 1)
where wt >nul 2>nul
if errorlevel 1 (
    start "Bookstore" cmd /c "mode con cols=80 lines=24 & java Main || pause"
) else (
    start "" wt -w new --size 80,24 --pos 0,0 -d . cmd /c "java Main || pause"
)
```

4.2 By hand

```
javac *.java
java Main
```

Use a terminal window at least 80 characters wide. Java class names are case-sensitive: `java Main` with a capital M.

5 System Design

5.1 File structure

```
Bookstore-JAVA/
├── Product.java      Item, Trackable, Promotable, Product, Book, Stationery
├── Bookstore.java    shop rules, saving and loading, class Sale
├── BookstoreUI.java  screens, colors, tables, receipt, questions
├── Main.java         start-up, menu, one method per task
├── run.bat           compiles and opens an 80 × 24 window
├── .gitignore        keeps .class files and bookstore.dat off GitHub
└── README.md         short guide for GitHub
```

File	Lines	Job
`Product.java`	130	The product model: what a product is and how each type describes itself.
`Bookstore.java`	130	The shop's rules and the data file. It never prints anything.
`BookstoreUI.java`	216	Draws every screen and reads every answer. It never changes data.
`Main.java`	141	The controller: shows the menu, decides which task runs and connects the UI with the bookstore.

5.2 How the parts work together

The program is split into layers so each class has one job. The user only talks to `BookstoreUI`; `Main` decides what happens; `Bookstore` checks the rules and saves; `Product` and `Sale` are the data.

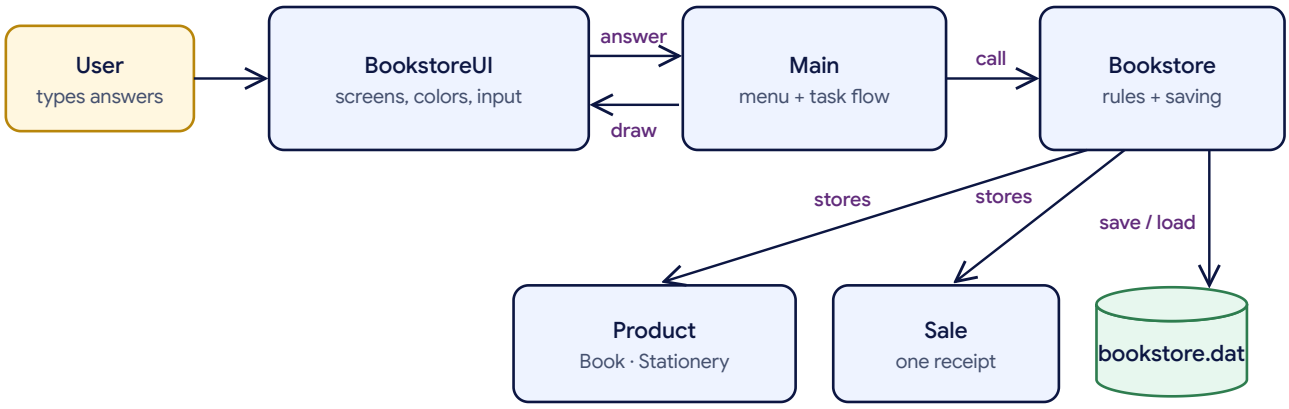


Figure 1. How the classes work together.

5.3 Class diagram

The product classes form the main inheritance hierarchy. Hollow triangles mean *extends* (solid line) or *implements* (dashed line).

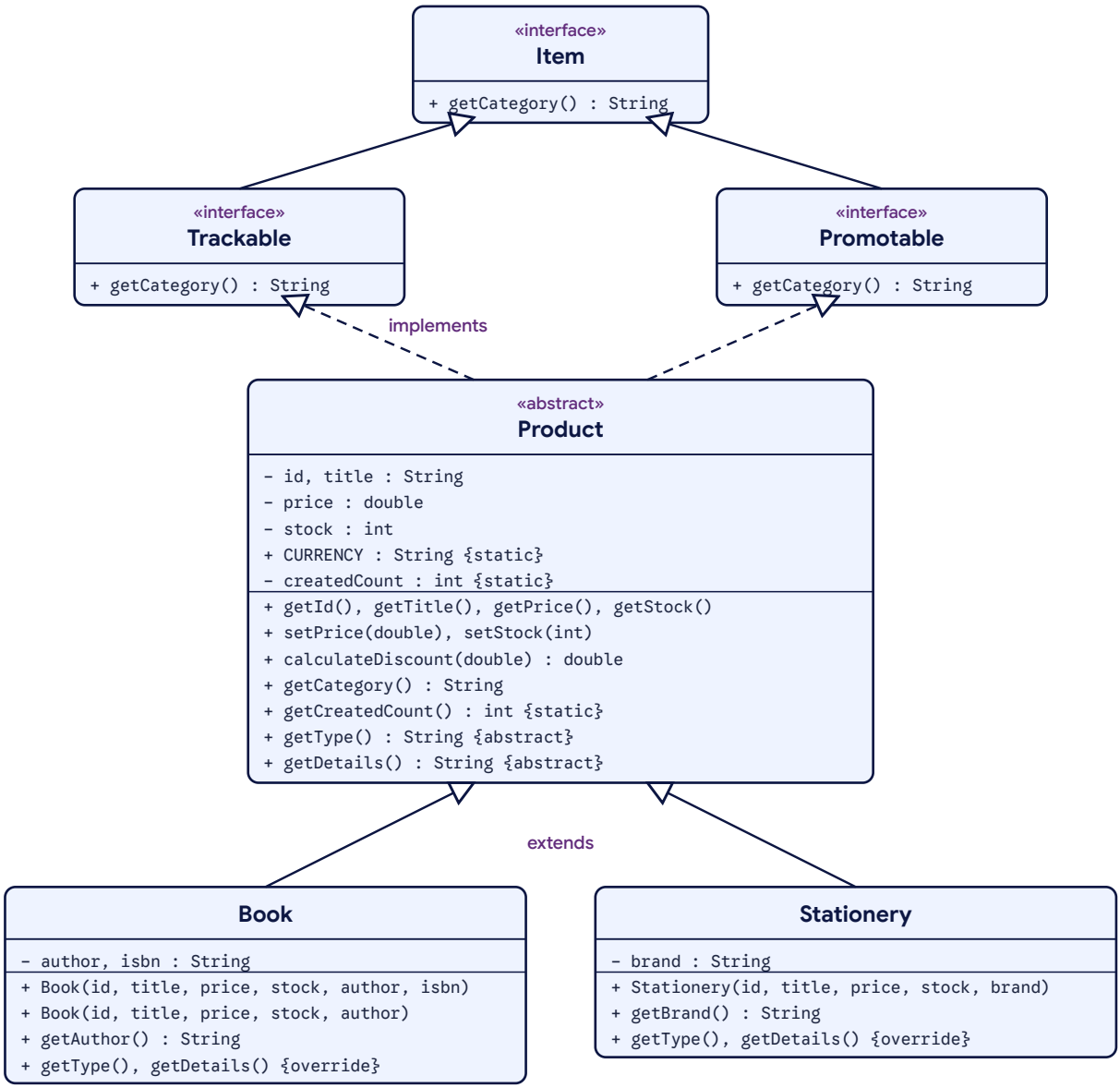


Figure 2. Class diagram of the product hierarchy (UML).

Class	Kind	Main members
Item, Trackable, Promotable	interfaces	`default getCategory()` (form the diamond, see 6.11)
Product	abstract class	id, title, price, stock; getters/setters; calculateDiscount; getType/getDetails (abstract)
Book	subclass of Product	author, isbn; two constructors; getAuthor; overrides getType/getDetails
Stationery	subclass of Product	brand; getBrand; overrides getType/getDetails
Bookstore	class	ArrayList of products and sales; find, get, search, add, restock, changePrice, total, sell, getSale, refund, save, load
Sale	class	id, date, productId, title, quantity, total, paid, refunded

Class	Kind	Main members
BookstoreUI	class	newScreen, askMenuChoice, askText/askNumber/askMoney/askLetter/askYes, showProducts, showProduct, showReceipt, showSales, showReport, message, error
Main	class (static)	main loop and one method per task: search, addProduct, restock, changePrice, sell, refund

5.4 Program flow

After start-up the program repeats one loop: draw a screen, ask for a choice, run the task, show the result. Any problem inside a task is caught in the loop and shown in red, so the program always returns to the menu.

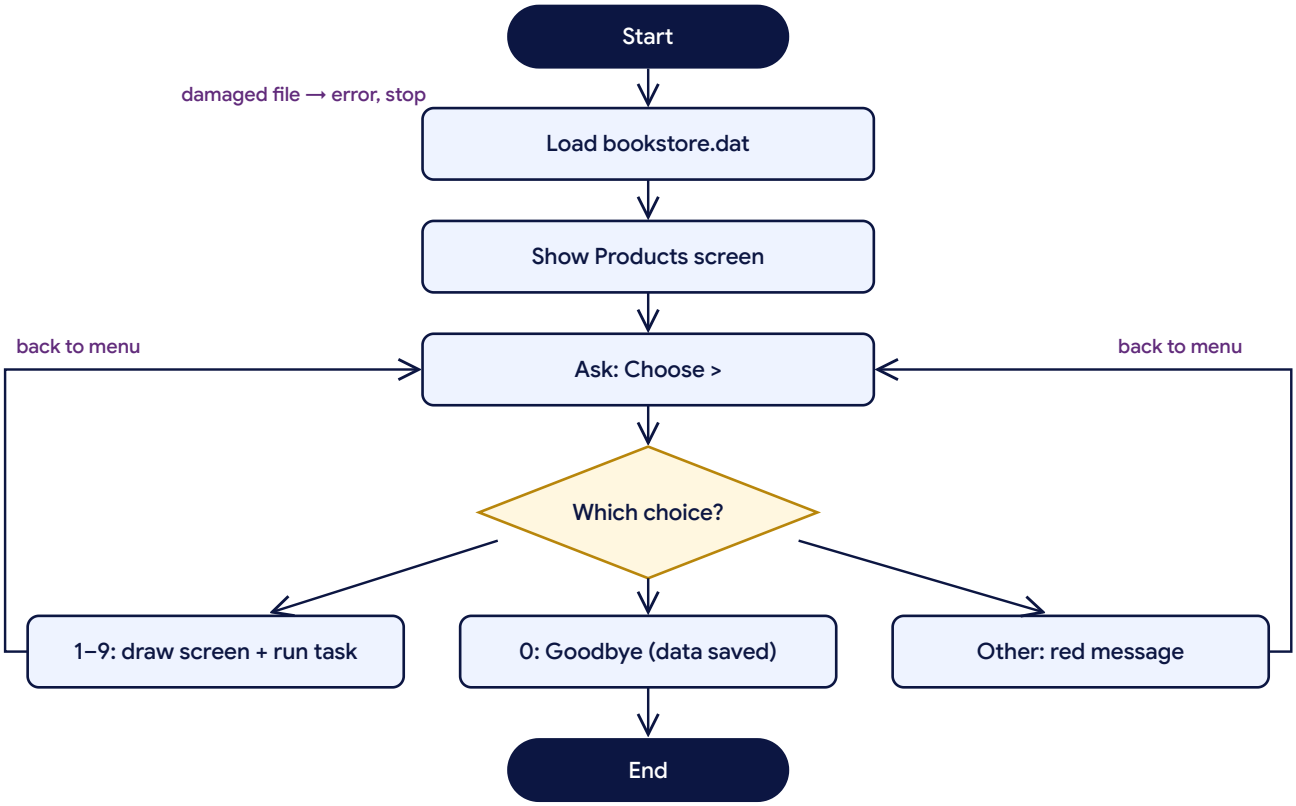


Figure 3. Main program loop.

Main.java main() — start-up and menu loop

```
8  public static void main(String[] args) {
9      Locale.setDefault(Locale.US); // always show prices as 450.50 (with a dot) on every
   computer
10     try {
11         store = Bookstore.load();
12     } catch (RuntimeException e) {
13         ui.error(e.getMessage());
14         System.exit(1); // stop with an error code, so run.bat keeps the window open to read
   this
15     }
16     ui.newScreen(1);
17     ui.showProducts(store.getProducts());
18     while (true) {
19         try {
20             String choice = ui.askMenuChoice();
21             switch (choice) {
22                 case "1": ui.newScreen(1); ui.showProducts(store.getProducts()); break;
23                 case "2": ui.newScreen(2); search(); break;
24                 case "3": ui.newScreen(3); addProduct(); break;
25                 case "4": ui.newScreen(4); restock(); break;
26                 case "5": ui.newScreen(5); changePrice(); break;
27                 case "6": ui.newScreen(6); sell(); break;
28                 case "7": ui.newScreen(7); refund(); break;
29                 case "8": ui.newScreen(8); ui.showSales(store.getSales()); break;
30                 case "9": ui.newScreen(9); ui.showReport(store); break;
31                 case "0": ui.goodbye(); return;
32                 default: ui.error("Please choose a number from the menu.");
33             }
34         } catch (RuntimeException e) {
35             ui.error(e.getMessage()); // show the problem in red, then go back to the menu
36         }
37     }
38 }
```

6 OOP Concepts in the Code

Each concept below is used by a real feature of the program. For each one: what it means, exactly where it is (file and line numbers), the real code, and what it does in this program.

#	Concept	Main location
6.1	Abstraction	Product.java — abstract class Product, interfaces
6.2	Encapsulation	Product.java — private fields, getters, checking setters
6.3	Inheritance	Product.java — Book and Stationery extend Product
6.4	Polymorphism	BookstoreUI.showProducts / showProduct — p.getDetails(), p.getType()
6.5	Static vs non-static	Product.java — CURRENCY, static block, createdCount, getCreatedCount()
6.6	Method overloading	Book constructors; Bookstore.restock(...)
6.7	Method overriding	Book / Stationery getType, getDetails; Product.getCategory
6.8	this keyword	Product constructor; Book(...) calls this(...)
6.9	super keyword	super(...) in Book/Stationery; Trackable.super.getCategory()
6.10	Subclass vs superclass reference	Main.addProduct — Book book ... Product product = book
6.11	Diamond problem resolution	Product.java — Item, Trackable, Promotable, getCategory()

6.1 Abstraction

Meaning: show only what is needed and hide the details. An abstract class is a blueprint that cannot be used to create objects directly.

Product.java lines 27–83

```
27  public abstract class Product implements Trackable, Promotable, Serializable {
 …
81  // ABSTRACT METHODS: every subclass must write its own version (overriding).
82  public abstract String getType();
83  public abstract String getDetails();
```

How it works here: `Product` is `abstract`, so `new Product(...)` does not compile. The shop only ever creates a `Book` or a `Stationery`. `Product` declares `getType()` and `getDetails()` without a body: every product must be able to describe itself, but each type decides how. The rest of the program (tables, search, receipts) only works with `Product` and never needs to know the exact type. The interfaces `Item`, `Trackable` and `Promotable` are also abstractions: they describe behaviour without any data.

> **Where in the code:** Product.java, line 27; Product.java, lines 82–83

6.2 Encapsulation

Meaning: keep an object's data private and allow access only through its own methods, which can protect the data.

Product.java lines 39–68

```
39  // NON-STATIC (instance) variables: every product object has its own copy.
40  private String id, title;
41  private double price;
42  private int stock;
…
56  public String getId() { return id; }
57  public String getTitle() { return title; }
58  public double getPrice() { return price; }
59  public int getStock() { return stock; }
…
61  // The setters protect the data: a negative value is refused.
62  public void setPrice(double price) {
63      if (price >= 0) this.price = price;
64  }
…
66  public void setStock(int stock) {
67      if (stock >= 0) this.stock = stock;
68  }
```

How it works here: `id`, `title`, `price` and `stock` are `private`, so no other class can change them directly. Other classes read them with getters and change them only with setters. The setters refuse a negative price or stock, so a product can never hold impossible values. `Book` (author, ISBN), `Stationery` (brand), `Bookstore` (the product and sales lists) and `BookstoreUI` (colors and the input scanner) also keep their fields private. `Sale` is a small data holder whose fields are read by the screens; a sale never changes except for its refunded flag, which only `Bookstore.refund` sets.

Where in the code: Product.java, lines 40–42; Product.java, lines 62–64; Product.java, lines 66–68

6.3 Inheritance

Meaning: a subclass receives the fields and methods of its superclass and adds its own.

Product.java lines 86–116

```
86  // INHERITANCE: Book is a subclass of the superclass Product.
87  class Book extends Product {
88      private static final long serialVersionUID = 1L;
89      private String author, isbn;
…
113 // INHERITANCE: Stationery is another subclass of Product.
114 class Stationery extends Product {
115     private static final long serialVersionUID = 1L;
116     private String brand;
```

How it works here: `Book` and `Stationery` are **subclasses** of the **superclass** `Product` ("a Book *is a* Product"). They inherit id, title, price, stock and every getter, setter and `calculateDiscount`, so this code is written only once. Each subclass adds only what is special about it: a book has an author and an ISBN, a stationery item has a brand. `Product` itself inherits `getCategory()` from its interfaces.

Where in the code: Product.java, line 87; Product.java, line 114

6.4 Polymorphism

Meaning: “many forms” — the same call does different things depending on the object. Java has two kinds:

- **Compile-time polymorphism** (method overloading): the compiler picks a method by its parameters — see 6.6.
- **Runtime polymorphism** (method overriding and dynamic dispatch): Java picks the method of the real object while the program runs.

BookstoreUI.java lines 135–162

```
135 for (Product p : list) {
136     String color = BODY;
137     String stock = p.getStock() + " ";
138     if (p.getStock() <= 3) {
139         color = YELLOW;
140         stock = p.getStock() + "!";
141         low++;
142     }
143     line(color, String.format(" %-9s %-21s %-29s %9.2f %6s",
144             fit(p.getId(), 9), fit(p.getTitle(), 21), fit(p.getDetails(), 29), p.getPrice(),
    stock));
…
156 public void showProduct(Product p) {
157     card(p.getType() + ": " + p.getTitle() + "  (" + p.getId() + ")");
158     card(p.getDetails());
159     card("Category: " + p.getCategory());
160     card(String.format("Price %.2f %s   |   In stock: %d", p.getPrice(), Product.CURRENCY,
    p.getStock()));
161     line(BODY, "");
162 }
```

How it works here: the product table loops over an `ArrayList<Product>`. For each `p` it calls `p.getDetails()`. The variable type is `Product`, but at run time Java calls `Book.getDetails()` for books (“Book by Robert C. Martin, ISBN …”) and `Stationery.getDetails()` for stationery (“Stationery by Pilot”). The product card does the same with `p.getType()` (“BOOK” or “STATIONERY”). One loop handles every product type, and a new type could be added without changing it.

> **Where in the code:** BookstoreUI.java, lines 135–144; BookstoreUI.java, lines 156–162

6.5 Static vs Non-Static

Meaning: a `static` member belongs to the class and is shared by all objects; a non-static member belongs to each object separately.

Product.java lines 30–53

```
30  // STATIC variables belong to the class and are shared by every product.
31  public static final String CURRENCY;
32  private static int createdCount = 0; // products created while the program runs (shown in the
    Report)
…
34  // STATIC BLOCK: runs once, when Java first loads the Product class.
35  static {
36      CURRENCY = "BDT";
37  }
…
44  public Product(String id, String title, double price, int stock) {
45      this.id = id; // THIS: this.id is the object's field, id is the parameter
46      this.title = title;
47      this.price = price;
48      this.stock = stock;
49      createdCount++;
50  }
…
52  // STATIC METHOD: called on the class itself, e.g. Product.getCreatedCount()
53  public static int getCreatedCount() { return createdCount; }
```

Kind	Member	In this program
Static variable	`public static final String CURRENCY`	One currency text (“BDT”) shared by every product; shown on the product card.
Static variable	`private static int createdCount`	Counts product objects created while the program runs (+1 in every constructor call).
Static block	`static { CURRENCY = "BDT"; }`	Runs once when Java first loads the Product class.
Static method	`Product.getCreatedCount()`	Called on the class, not an object; shown in the Report as “New products added this session”.
Static members	`Main.main, Main.sell, Main.store, Main.ui`	Belong to the Main class; no Main object is ever created.
Non-static variables	`price, stock, author, brand …`	Every product object has its own values.
Non-static methods	`getPrice(), setStock(), all BookstoreUI methods`	Work on one object’s own data.

Saved products are read back from the file without calling a constructor, so `createdCount` counts only products added in the current session — that is exactly what the Report shows.

> **Where in the code:** Product.java, lines 31–32; Product.java, lines 35–37; BookstoreUI.java, line 212; BookstoreUI.java, line 160

6.6 Method Overloading

Meaning: several methods (or constructors) with the same name but different parameter lists. The compiler chooses one from the arguments.

Product.java lines 91–101

```
91  // CONSTRUCTOR OVERLOADING 1: a book with an ISBN.
92  public Book(String id, String title, double price, int stock, String author, String isbn) {
93      super(id, title, price, stock); // SUPER: runs the Product constructor first
94      this.author = author;
95      this.isbn = isbn;
96  }
…
98  // CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor above.
99  public Book(String id, String title, double price, int stock, String author) {
100     this(id, title, price, stock, author, "N/A");
101 }
```

Bookstore.java lines 45–54

```
45  // METHOD OVERLOADING: restock by product ID, or by the product object itself.
46  public void restock(String id, int quantity) {
47      restock(get(id), quantity);
48  }
…
50  public void restock(Product p, int quantity) {
51      if (quantity < 1) throw new RuntimeException("Quantity must be at least 1.");
52      p.setStock(p.getStock() + quantity);
53      save();
54  }
```

Main.java lines 77–96

```
77  String isbn = ui.askText("ISBN (type - if it has none)");
78  Book book; // SUBCLASS reference
79  if (isbn.equals("-")) book = new Book(id, title, price, stock, author); // overloaded
    constructor
80  else book = new Book(id, title, price, stock, author, isbn);
…
96  store.restock(p.getId(), ui.askNumber("Quantity to add", 1, 1000000)); // overloaded
    restock(String, int)
```

How it works here: `Book` has two constructors. When the user types an ISBN, the 6-parameter constructor is called; when the user types "-" (many local books have no ISBN), the 5-parameter one is called. `Bookstore` has two `restock` methods: one takes a product ID, the other a `Product` object. The Restock screen calls `restock(String, int)`, which finds the product and calls `restock(Product, int)` to do the work.

Where in the code: Product.java, lines 92–96; Product.java, lines 99–101; Bookstore.java, lines 46–54

6.7 Method Overriding

Meaning: a subclass gives its own version of a method it received from its superclass or an interface. `@Override` asks the compiler to check that a matching method really exists.

Product.java lines 105–129

```java
// METHOD OVERRIDING: Book's own versions of Product's abstract methods.
@Override
public String getType() { return "BOOK"; }

@Override
public String getDetails() { return "Book by " + author + ", ISBN " + isbn; }
…
public String getType() { return "STATIONERY"; }

@Override
public String getDetails() { return "Stationery by " + brand; }
```

How it works here: `Book` and `Stationery` override the abstract `getType()` and `getDetails()` of `Product`. These overrides are what make the runtime polymorphism in 6.4 work. `Product` also overrides `getCategory()` from its interfaces to solve the diamond problem (6.11), and `Trackable` / `Promotable` override `Item`'s default method.

Where in the code: Product.java, lines 105–110; Product.java, lines 125–129

6.8 The this Keyword

Meaning: `this` refers to the current object. It is used to tell a field apart from a parameter with the same name, and `this(...)` calls another constructor of the same class (constructor chaining).

Product.java lines 44–101

```java
public Product(String id, String title, double price, int stock) {
    this.id = id; // THIS: this.id is the object's field, id is the parameter
    this.title = title;
    this.price = price;
    this.stock = stock;
    createdCount++;
}
…
// CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor above.
public Book(String id, String title, double price, int stock, String author) {
    this(id, title, price, stock, author, "N/A");
}
```

How it works here: in `this.id = id` the left side is the object's field and the right side is the constructor parameter. The second `Book` constructor starts with `this(id, title, price, stock, author, "N/A")`: it hands the work to the first constructor and fills the missing ISBN with "N/A", so the setup code is written only once.

Where in the code: Product.java, line 45; Product.java, line 100

6.9 The super Keyword

Meaning: `super` refers to the parent. `super(...)` calls the superclass constructor; `Interface.super.method()` calls a parent interface's version of a method.

Product.java lines 118–79

```
118 public Stationery(String id, String title, double price, int stock, String brand) {
119     super(id, title, price, stock); // SUPER: runs the Product constructor first
120     this.brand = brand;
121 }
 …
 77 public String getCategory() {
 78     return Trackable.super.getCategory() + " & " + Promotable.super.getCategory();
 79 }
```

How it works here: a `Stationery` (and a `Book`) first calls `super(id, title, price, stock)`, so the `Product` constructor stores the shared fields and increases the static counter; then the subclass stores its own field. In `getCategory()`, `Trackable.super` and `Promotable.super` call each parent interface's version.

Where in the code: Product.java, line 93; Product.java, line 78

6.10 Subclass Reference vs Superclass Reference

Meaning: a **subclass reference** (`Book book`) can use everything a Book has. A **superclass reference** (`Product product`) can point to any subclass object — this is **upcasting** — but only sees the methods declared in `Product`.

Main.java lines 73–89

```
73 Product product; // SUPERCLASS reference: can hold a Book or a Stationery
74 String extra;
75 if (type == 'B') {
76     String author = ui.askText("Author");
77     String isbn = ui.askText("ISBN (type - if it has none)");
78     Book book; // SUBCLASS reference
79     if (isbn.equals("-")) book = new Book(id, title, price, stock, author); // overloaded
   constructor
80     else book = new Book(id, title, price, stock, author, isbn);
81     extra = " by " + book.getAuthor(); // getAuthor() works only through the Book reference
82     product = book;                    // UPCASTING: the Book is stored as a Product
83 } else {
84     Stationery item = new Stationery(id, title, price, stock, ui.askText("Brand"));
85     extra = " from " + item.getBrand();
86     product = item;
87 }
88 store.add(product);
89 showResult(3, "Added " + product.getType() + " \"" + title + "\"" + extra + ".");
```

How it works here: while a book is being created it is held in `Book book`, so `book.getAuthor()` can be called (`Product` has no such method). Then `product = book` stores the same object in a `Product` variable. From then on the program treats it like any other product: `store.add(product)` puts it in the `ArrayList<Product>`, and `product.getType()` still runs Book's version. The green message "Added BOOK "..." by ..." uses both references.

Where in the code: Main.java, lines 73–89; Bookstore.java, line 9

6.11 Diamond Problem Resolution

Meaning: Java does not allow a class to extend two classes (`class C extends A, B`), which avoids the diamond problem for classes. With interfaces it can still happen: if two interfaces give different `default` versions of the same method and a class implements both, Java cannot know which one to use, so it refuses to compile until the class decides.

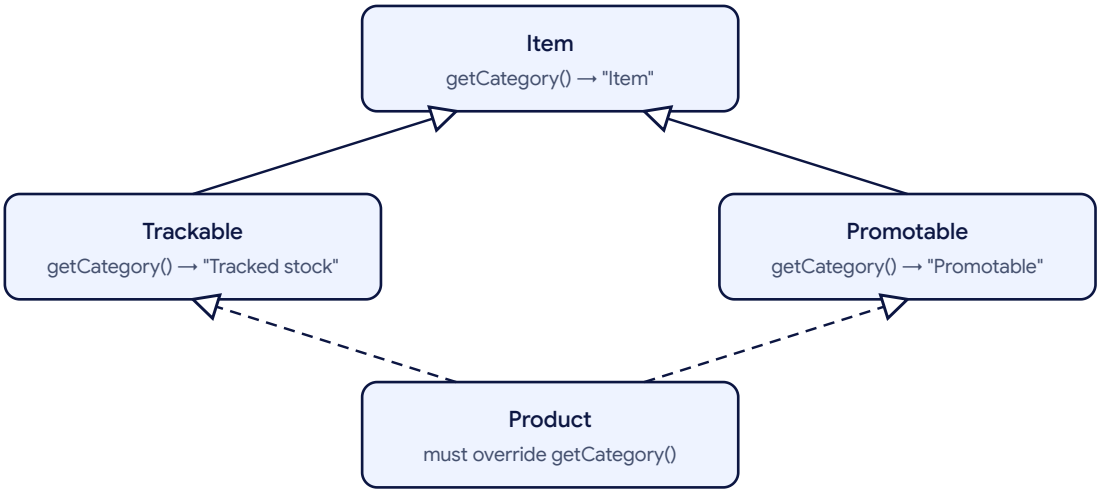


Figure 4. The interface diamond: Item at the top, Trackable and Promotable in the middle, Product at the bottom.

Product.java lines 3–79

```
3  // DIAMOND PROBLEM: Trackable and Promotable both extend Item, and both give their own
4  // default getCategory(). Product implements both, so Java forces Product to choose.
5  //
6  //                Item
7  //              /      \
8  //       Trackable    Promotable
9  //              \      /
10 //              Product
11 interface Item {
12     default String getCategory() { return "Item"; }
13 }
14
15 interface Trackable extends Item {
16     @Override
17     default String getCategory() { return "Tracked stock"; }
18 }
19
20 interface Promotable extends Item {
21     @Override
22     default String getCategory() { return "Promotable"; }
23 }
…
27 public abstract class Product implements Trackable, Promotable, Serializable {
…
75 // DIAMOND PROBLEM SOLVED: we say exactly which parent versions to use.
76 @Override
77 public String getCategory() {
78     return Trackable.super.getCategory() + " & " + Promotable.super.getCategory();
79 }
```

How it works here: `Trackable` and `Promotable` both extend `Item` and both override `getCategory()`. `Product` implements both, which forms the diamond. `Product` overrides `getCategory()` and decides exactly what to do: it calls `Trackable.super.getCategory()` and `Promotable.super.getCategory()` and joins them. The result, “Tracked stock & Promotable”, is shown on the product card when restocking, changing a price or selling (Figures 10, 12 and 14).

> **Where in the code:** Product.java, lines 11–23; Product.java, lines 77–79; BookstoreUI.java, line 159

6.12 Other Java features used

Feature	Where	Purpose
Classes, objects and constructors	All files	Every product and sale is an object created with a constructor
Interfaces with default methods	Product.java	Item, Trackable, Promotable (diamond problem)
Collections and generics	`ArrayList<Product>, ArrayList<Sale>`	Store any number of products and sales

Feature	Where	Purpose
Exceptions	`throw new RuntimeException(...)`, `try { … } catch`	Bookstore rules report problems; Main shows them in red
try-with-resources	Bookstore.save / load	Files are always closed, even after an error
Serialization	`implements Serializable`	Save and load the whole bookstore in one step
Loops and switch	Main, BookstoreUI	Menu loop, ask-again loops, tables
Formatting	`String.format`	Aligned columns and money with two decimals
Date and time	`LocalDate, LocalDateTime`	Receipt date and the clock in the screen bar
Lambda expression	`new Thread(() -> …)`	Restores terminal colors when the program ends, even after Ctrl+C

7 User Interface and Screens

7.1 Screen layout

Every screen has the same layout, so the user always knows where to look. The whole program fits in an 80 × 24 character window.

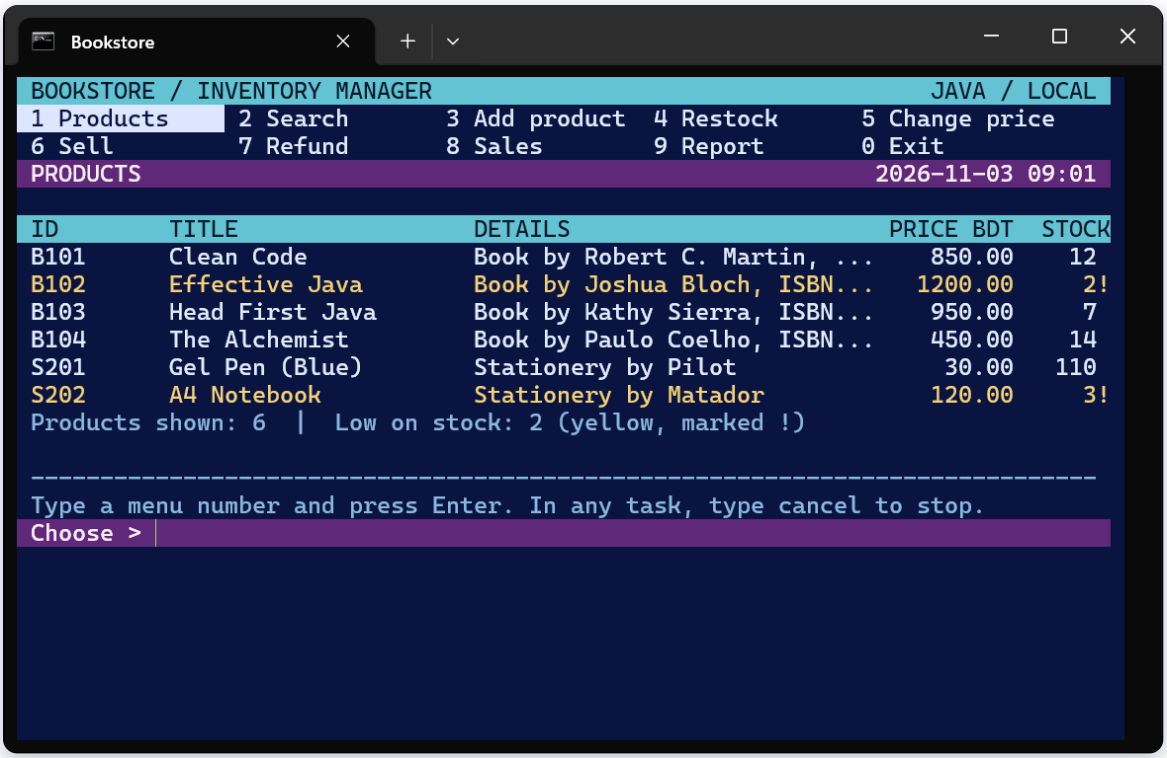


Figure 5. Products screen — the home screen of the program.

Part	Color	Content
Title bar	cyan	Program name and “JAVA / LOCAL”
Menu tabs	white = current	Keys 1–9 and 0; the current screen is highlighted
Screen bar	purple	Name of the screen and the current date and time
Body	navy	Tables, product card, receipt, report
Low stock / refunded	yellow	Rows that need attention
Success message	green	Result of a task, e.g. “Sale saved.”
Error message	red	What was wrong and what to do
Input bar	purple	The current question; the answer is typed here

7.2 How the colors are drawn

Terminals understand *ANSI escape codes*: special texts that start with `\033[`. `38;2;R;G;B` sets the text color and `48;2;R;G;B` the background color. The UI keeps each color in a named constant and prints every line through one small helper, `line()`.

BookstoreUI.java lines 10–38

```
10  private static final String BODY = "\033[38;2;229;237;255;48;2;12;22;66m";     // light text on
    navy
11  private static final String DIM = "\033[38;2;140;189;225;48;2;12;22;66m";      // soft blue on
    navy
12  private static final String YELLOW = "\033[38;2;248;212;123;48;2;12;22;66m";   // yellow on
    navy
13  private static final String CYAN = "\033[38;2;9;25;47;48;2;103;199;211m";      // dark on cyan
14  private static final String PURPLE = "\033[38;2;255;241;255;48;2;97;43;123m"; // white on
    purple
15  private static final String WHITE = "\033[38;2;12;22;66;48;2;222;233;255m";    // navy on white
16  private static final String GREEN = "\033[38;2;13;43;31;48;2;132;210;164m";    // dark on green
17  private static final String RED = "\033[38;2;255;239;239;48;2;137;35;50m";     // white on red
18  private static final String RESET = "\033[0m";                                 // terminal's
    own colors
…
29  // Prints one line: the text in its color, then the rest of the line in navy.
30  private void line(String color, String text) {
31      System.out.println(color + text + BODY + "\033[K");
32  }
…
34  // Shortens text that is too long for a table column, e.g. "A Very Long Ti..."
35  private String fit(String text, int width) {
36      if (text.length() <= width) return text;
37      return text.substring(0, width - 3) + "...";
38  }
```

BookstoreUI.java lines 40–57

```
40  // Clears the screen (and old screens above it), then draws the title bar, menu tabs and screen
    name.
41  public void newScreen(int tab) {
42      System.out.print(BODY + "\033[2J\033[3J\033[H");
43      line(CYAN, String.format(" %-60s%17s ", "BOOKSTORE / INVENTORY MANAGER", "JAVA / LOCAL"));
44      String tabs = "";
45      for (int n = 1; n <= 10; n++) {
46          int number = n % 10; // 1, 2, ... 9, then 0 for Exit
47          String color = (number == tab) ? WHITE : BODY;
48          tabs = tabs + color + String.format(" %d %-12s", number, MENU[number]) + BODY;
49          if (n == 5 || n == 10) {
50              line(BODY, tabs);
51              tabs = "";
52          }
53      }
54      String time = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm"));
55      line(PURPLE, String.format(" %-60s%17s ", MENU[tab].toUpperCase(), time));
56      line(BODY, "");
57  }
```

`newScreen` clears the window, draws the title bar, builds the two rows of menu tabs (the current one in white) and the purple screen bar with the clock. `fit()` shortens long text with "..." so table columns never break the layout.

7.3 Products and Search

Choosing **2** asks for a search text and shows only the matching products. Search looks at the ID, title and details, ignoring upper/lower case.

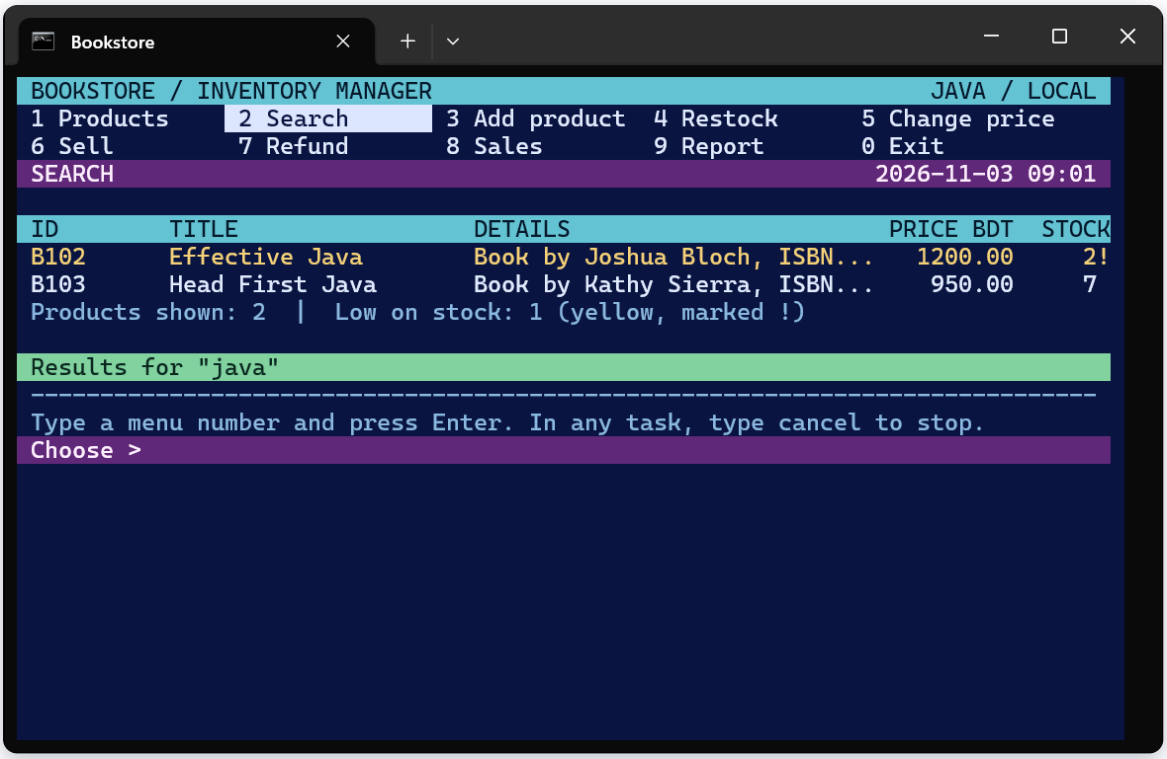


Figure 6. Search for "java": two books match.

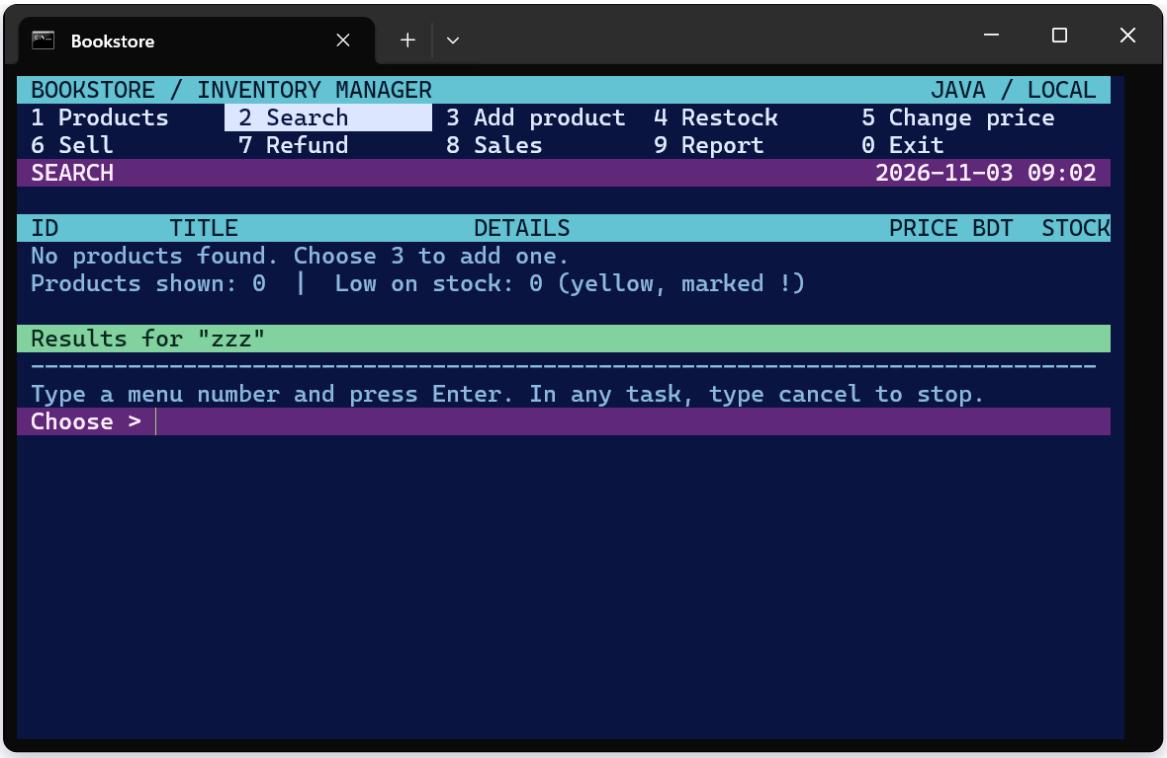


Figure 7. A search with no results.

Bookstore.java lines 30–37

```java
public ArrayList<Product> search(String text) {
    ArrayList<Product> found = new ArrayList<>();
    for (Product p : products) {
        String all = p.getId() + " " + p.getTitle() + " " + p.getDetails();
        if (all.toLowerCase().contains(text.toLowerCase())) found.add(p);
    }
    return found;
}
```

7.4 Add product

The Add screen asks one question at a time. Each wrong answer is explained in red and asked again; earlier answers stay. Figure 8 shows three mistakes being handled: a wrong type letter, an ID that already exists and a price that is not a number.

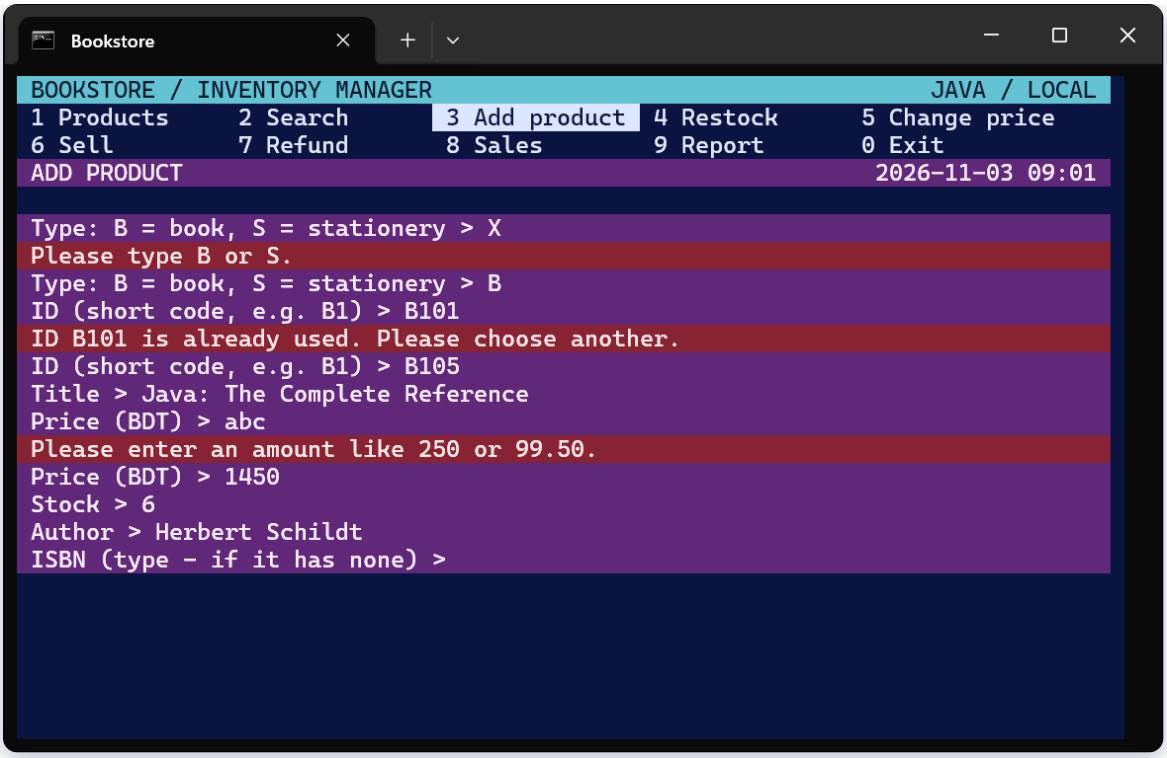


Figure 8. Adding a book, with three wrong answers handled.

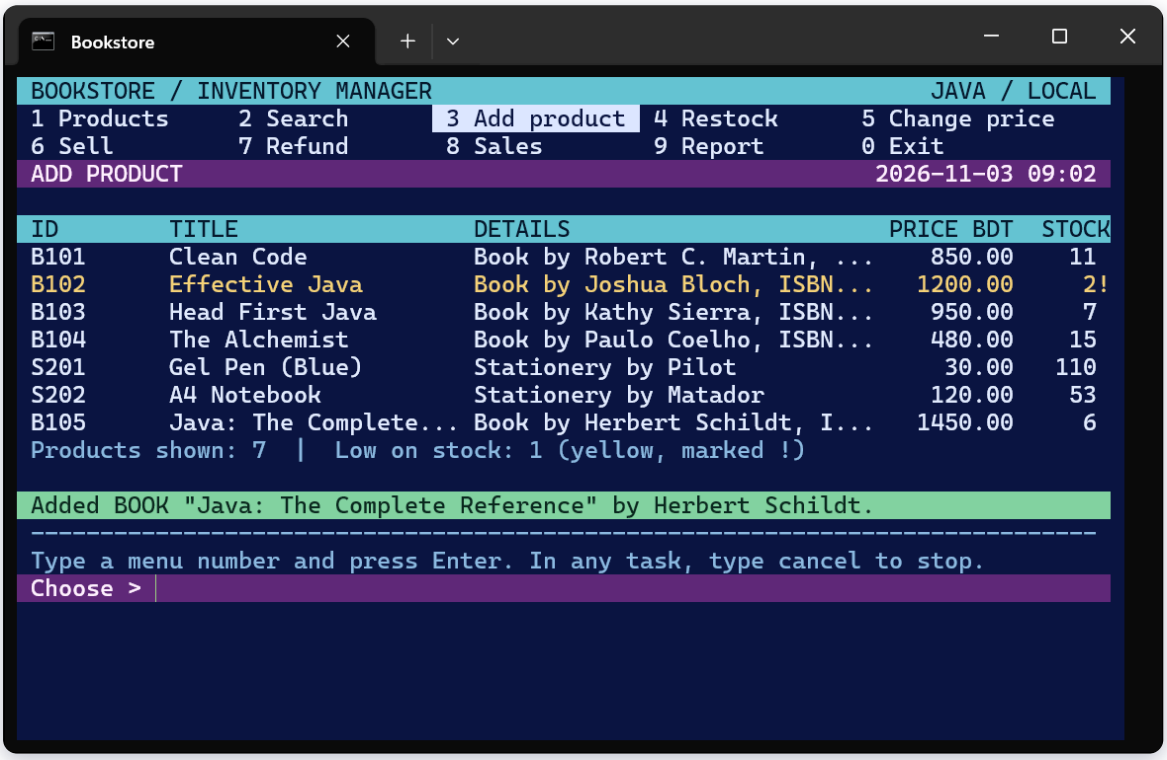


Figure 9. After adding: the table shows the new book (B105) and a green message.

7.5 Restock and Change price

Both screens first show the product table, then ask for a product ID. A wrong ID is explained and asked again. The white product card then shows the type, details, the diamond-problem category, the price and the current stock before the new value is entered.

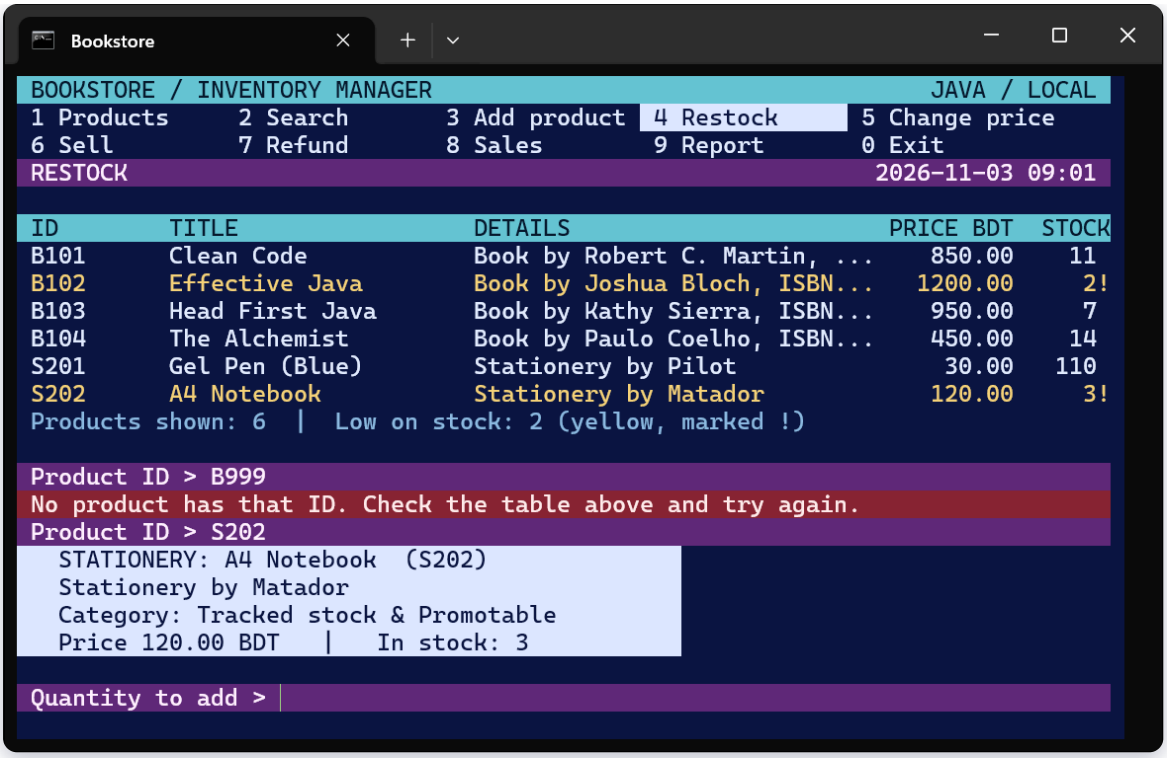


Figure 10. Restock: a wrong ID (B999) is refused, then the card for S202 is shown.

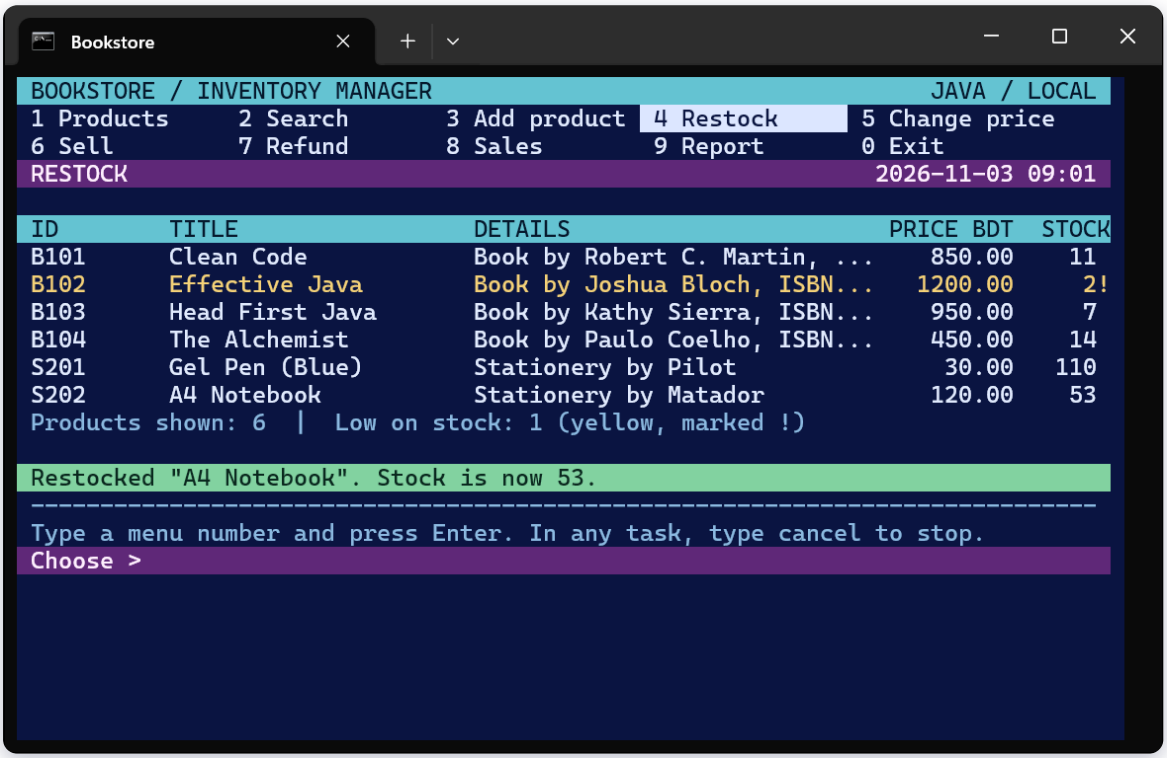


Figure 11. After restocking: the A4 Notebook has 53 units and is no longer yellow.

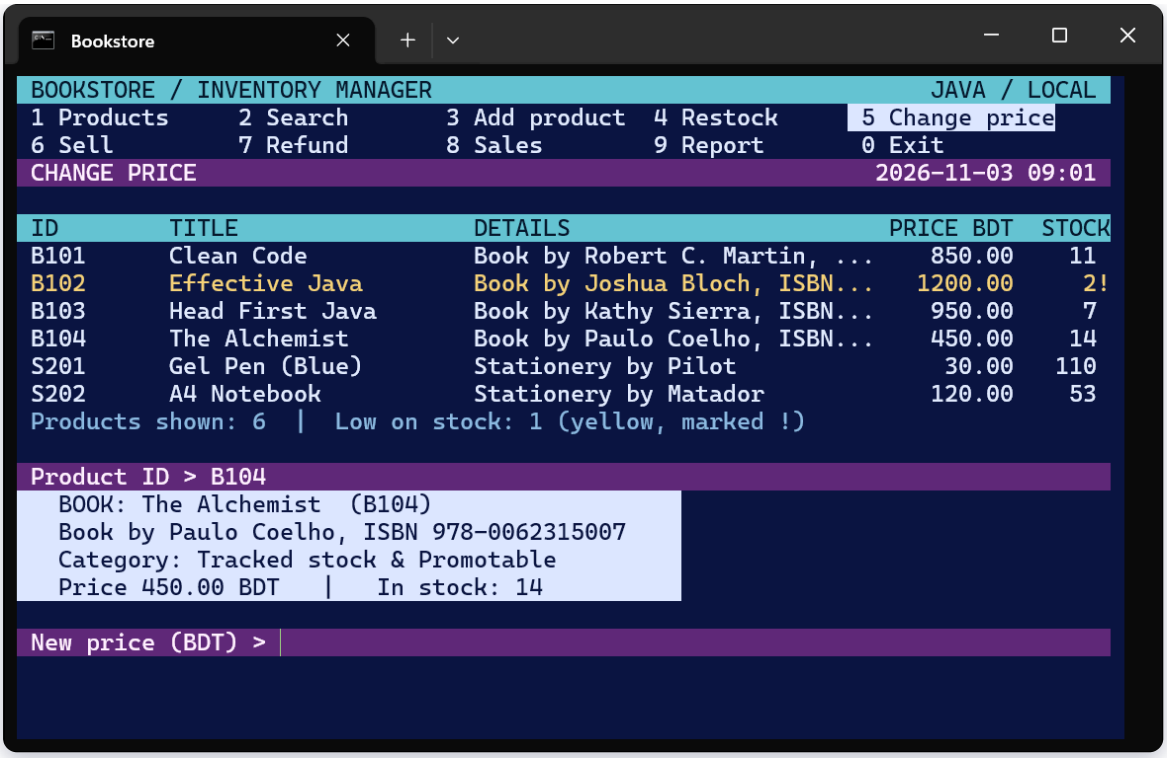


Figure 12. Change price: the card shows the current price before the new one is typed.

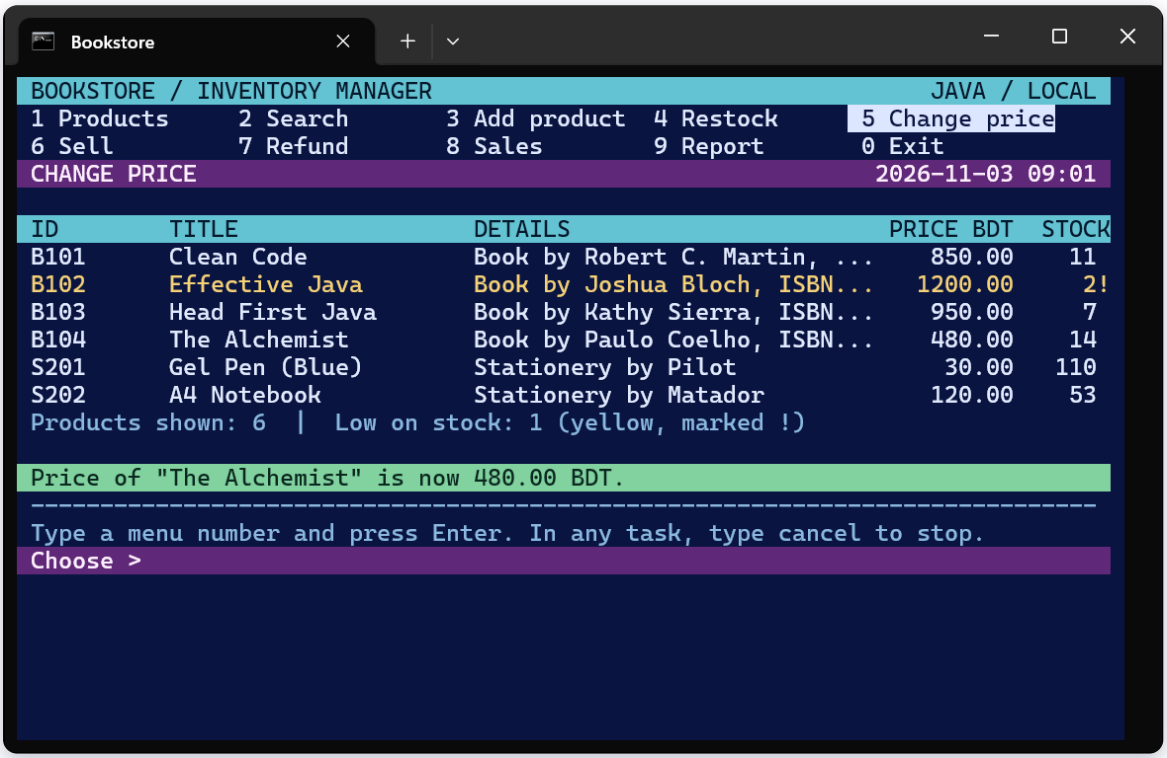


Figure 13. The new price is saved and shown in the table.

7.6 Sell

The Sell screen shows the table, then the card of the chosen product. The quantity question shows the allowed range (1 to the stock). After the member question the total is shown in green, and the cash must be at least the total. The screen is then redrawn with a paper-style receipt and the change to give back.

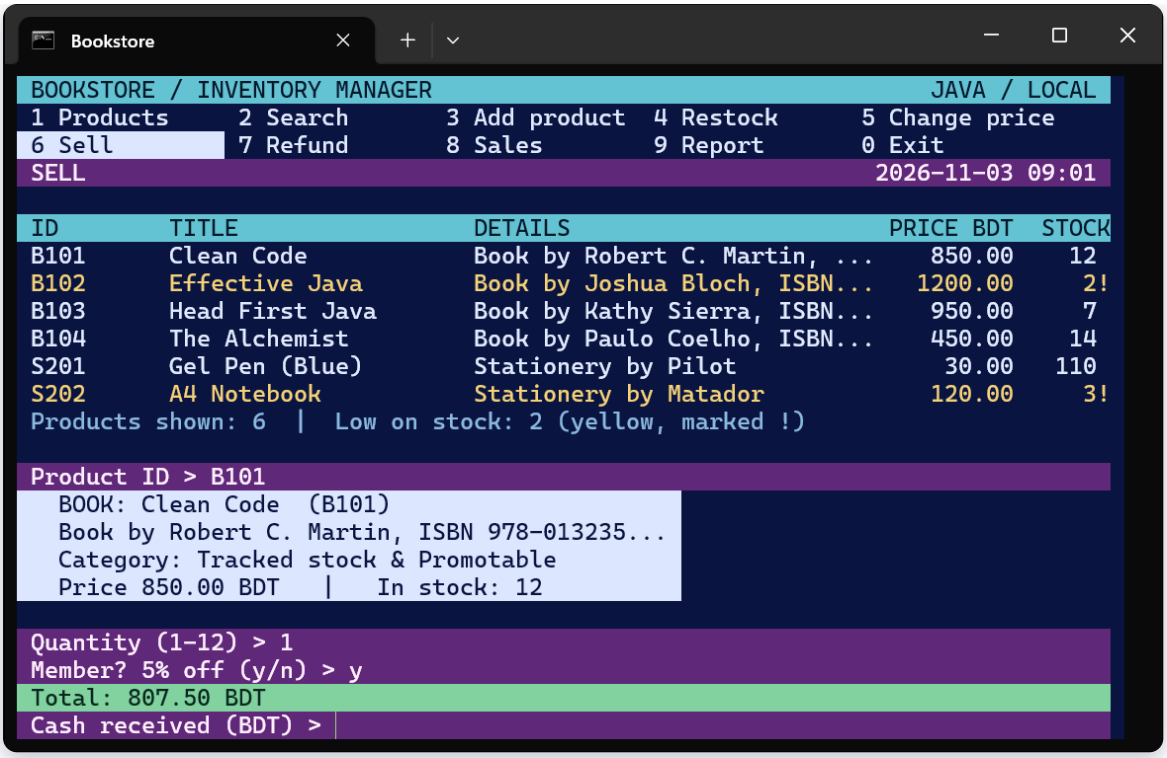


Figure 14. Selling one copy of Clean Code to a member: 850.00 − 5% = 807.50 BDT.

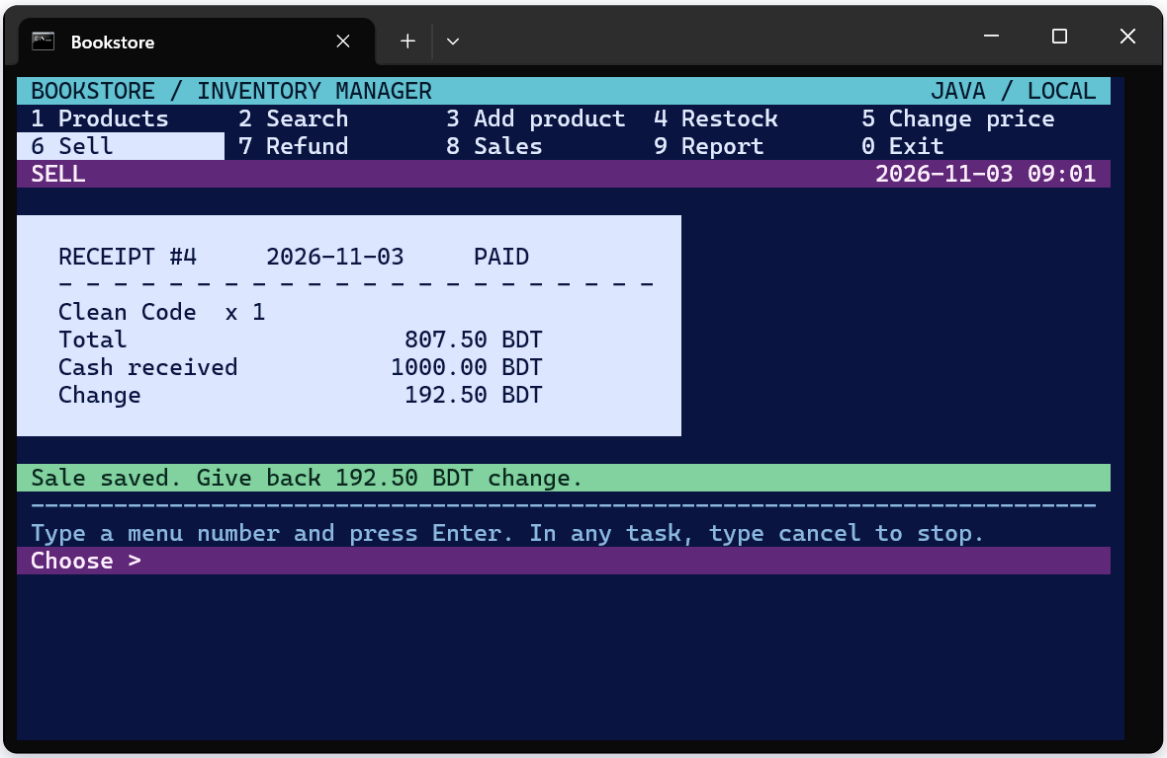


Figure 15. The receipt after the sale, with the change to give back.

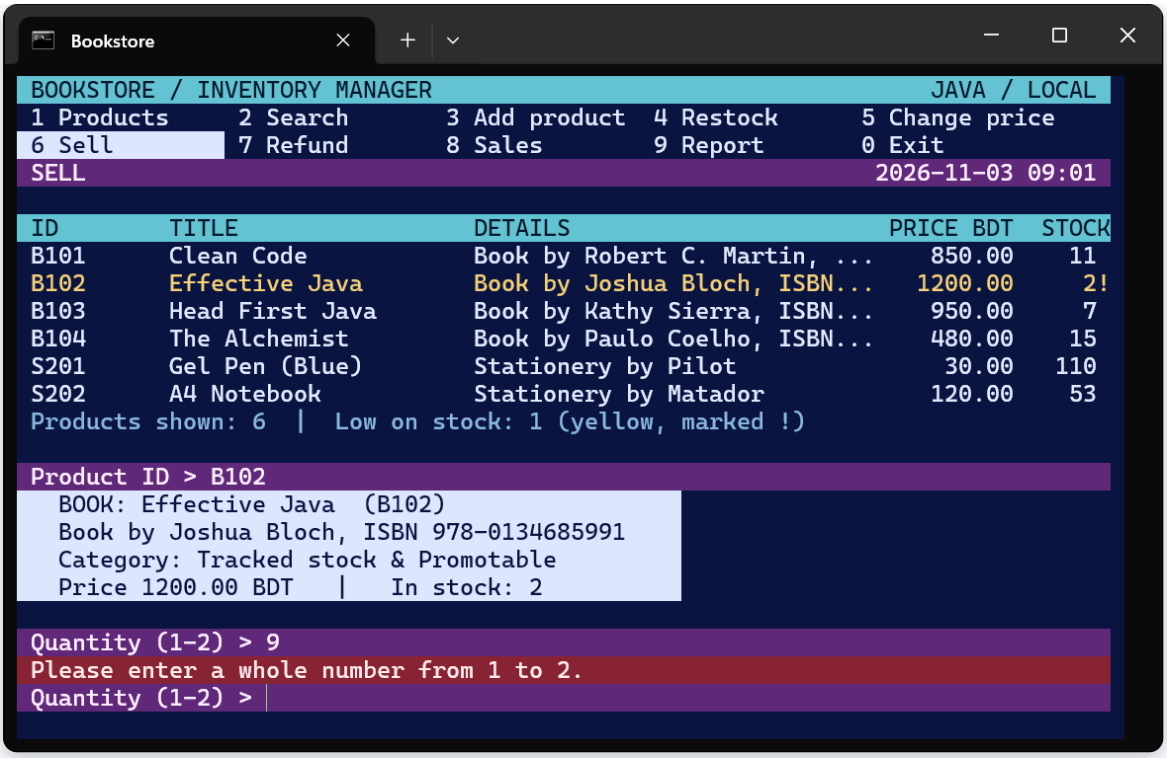


Figure 16. Only 2 copies are in stock, so a quantity of 9 is refused and asked again.

Main.java lines 108–124

```
108 static void sell() {
109     ui.showProducts(store.getProducts());
110     Product p = askProduct();
111     while (p.getStock() == 0) {
112         ui.error("\"" + p.getTitle() + "\" is out of stock. Choose another product.");
113         p = askProduct();
114     }
115     ui.showProduct(p);
116     int quantity = ui.askNumber("Quantity (1-" + p.getStock() + ")", 1, p.getStock());
117     boolean member = ui.askYes("Member? 5% off");
118     double total = store.total(p, quantity, member);
119     ui.message(String.format("Total: %.2f BDT", total));
120     Sale sale = store.sell(p, quantity, member, ui.askMoney("Cash received (BDT)", total));
121     ui.newScreen(6);
122     ui.showReceipt(sale);
123     ui.message(String.format("Sale saved. Give back %.2f BDT change.", sale.paid -
    sale.total));
124 }
```

7.7 Refund

The Refund screen lists all receipts. After choosing a receipt number, its receipt is shown and the user confirms. The stock goes back and the receipt is marked REFUNDED, so it cannot be refunded again.

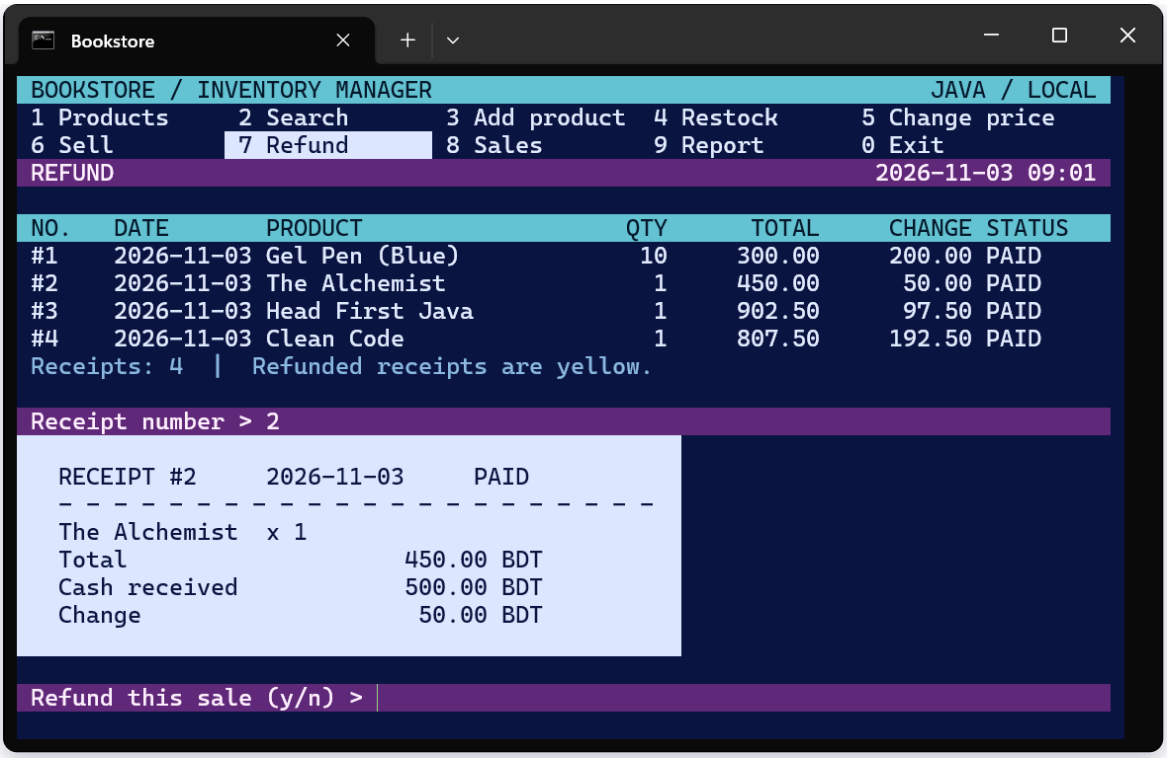


Figure 17. Choosing receipt #2 and confirming the refund.

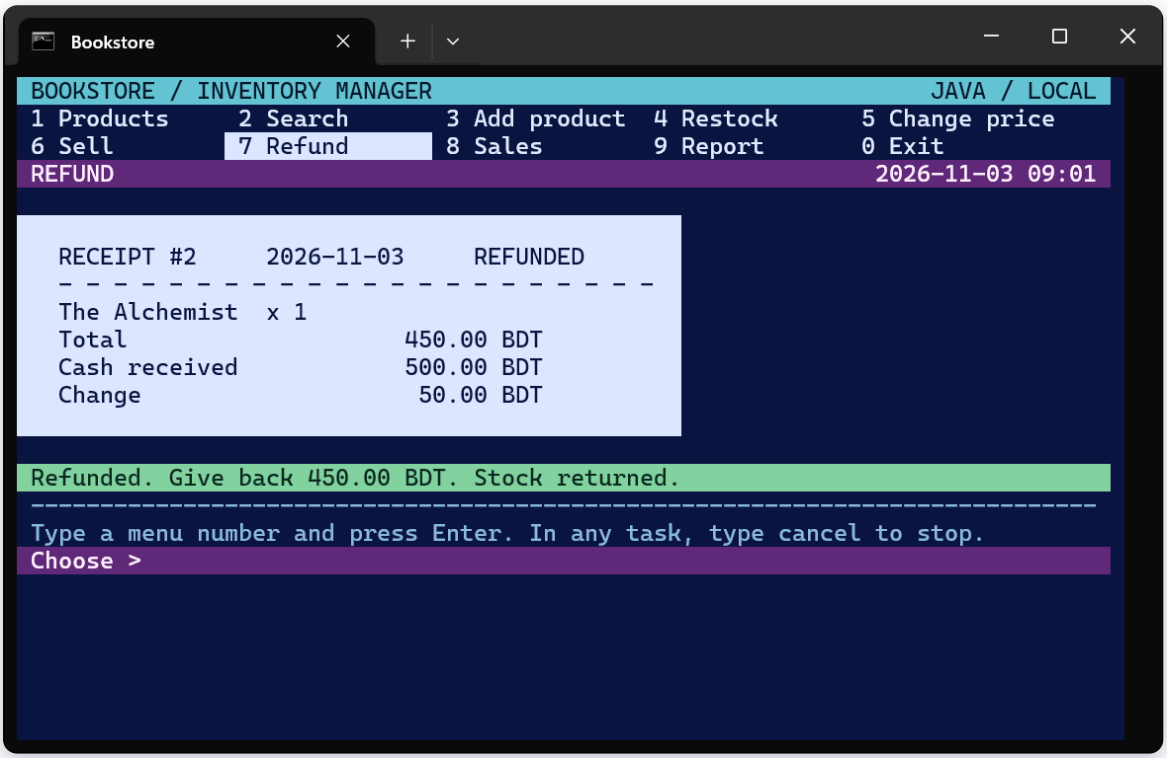


Figure 18. After the refund: the receipt is marked REFUNDED.

7.8 Sales history and Report

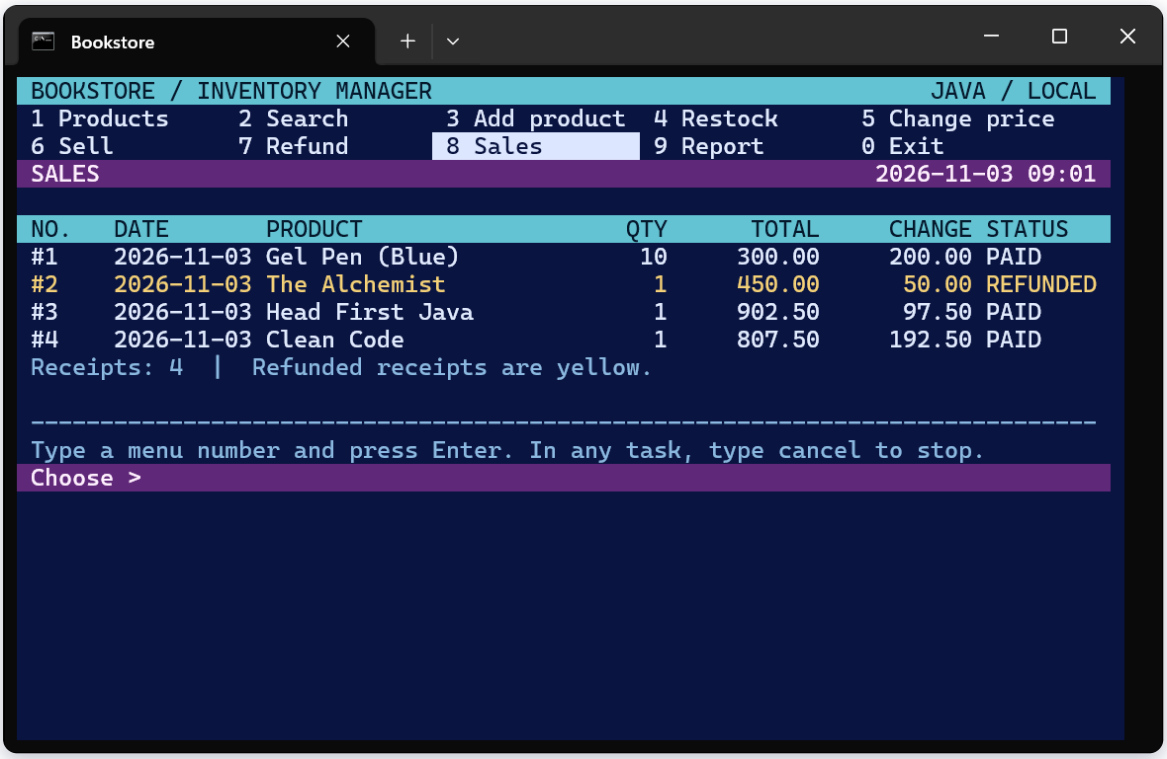


Figure 19. Sales history: the refunded receipt is shown in yellow.

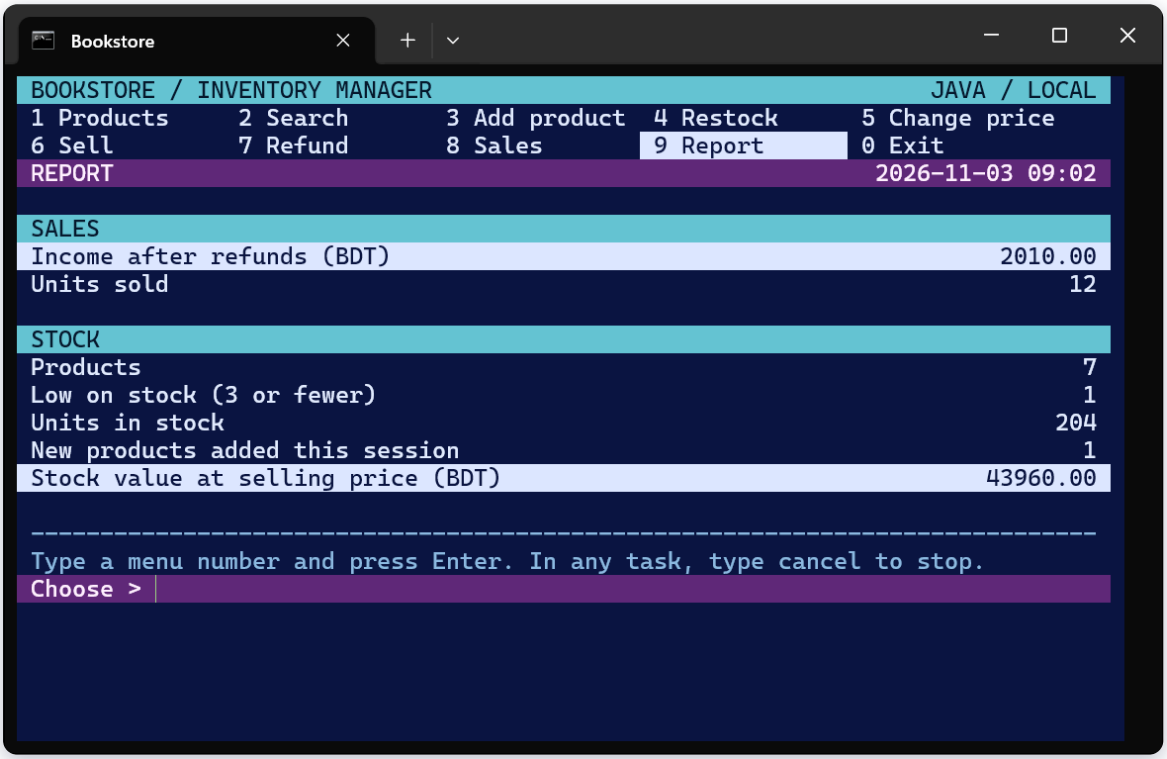


Figure 20. Report: income after refunds, units, low stock, products added this session and stock value.

Income in the report adds only receipts that were not refunded: 300.00 + 902.50 + 807.50 = 2010.00 BDT (receipt #2 was refunded). Stock value is the sum of price × stock for every product.

BookstoreUI.java lines 190–215

```java
public void showReport(Bookstore store) {
    double income = 0, stockValue = 0;
    int unitsSold = 0, unitsInStock = 0, lowStock = 0;
    for (Sale s : store.getSales()) {
        if (!s.refunded) {
            income = income + s.total;
            unitsSold = unitsSold + s.quantity;
        }
    }
    for (Product p : store.getProducts()) {
        unitsInStock = unitsInStock + p.getStock();
        stockValue = stockValue + p.getPrice() * p.getStock();
        if (p.getStock() <= 3) lowStock++;
    }
    line(CYAN, String.format(" %-78s", "SALES"));
    line(WHITE, String.format(" %-58s%19.2f ", "Income after refunds (BDT)", income));
    line(BODY, String.format(" %-58s%19d", "Units sold", unitsSold));
    line(BODY, "");
    line(CYAN, String.format(" %-78s", "STOCK"));
    line(BODY, String.format(" %-58s%19d", "Products", store.getProducts().size()));
    line(BODY, String.format(" %-58s%19d", "Low on stock (3 or fewer)", lowStock));
    line(BODY, String.format(" %-58s%19d", "Units in stock", unitsInStock));
    line(BODY, String.format(" %-58s%19d", "New products added this session",
Product.getCreatedCount()));
    line(WHITE, String.format(" %-58s%19.2f ", "Stock value at selling price (BDT)",
stockValue));
    line(BODY, "");
}
```

7.9 Cancelling a task

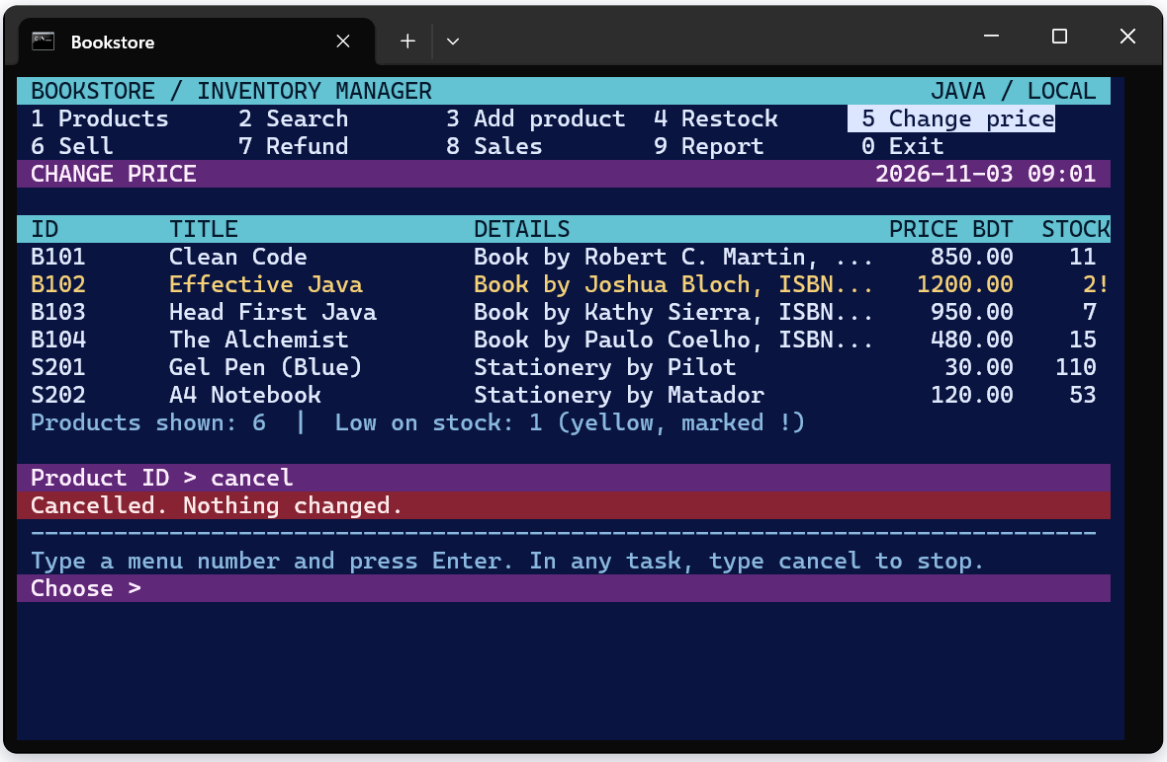


Figure 21. Typing cancel stops the task; nothing is changed.

8 Input Validation and Error Handling

Two levels protect the data. The **user interface** asks again until an answer is valid, so the user never loses earlier answers. The **Bookstore** class checks its rules again before changing anything, so even a programming mistake cannot store wrong data.

Question / action	Rule	Message shown
Menu choice	Must be 0–9	Please choose a number from the menu.
Product type	B or S (first letter)	Please type B or S.
New product ID	Must not exist yet	ID … is already used. Please choose another.
Price	A number from 0 to 1,000,000,000 (rounded to 2 decimals)	Please enter an amount like 250 or 99.50.
Stock	Whole number 0–1,000,000	Please enter a whole number from 0 to 1000000.
Product ID (restock, price, sell)	Must exist	No product has that ID. Check the table above and try again.
Selling an empty product	Stock must be above 0	"…" is out of stock. Choose another product.
Sell quantity	1 to the stock	Please enter a whole number from 1 to N.
Restock quantity	1 to 1,000,000	Please enter a whole number from 1 to 1000000.
Yes/no questions	y/yes or n/no	Please type Y or N.
Cash received	At least the total	Please enter at least … BDT.
Refund with no sales	There must be a sale	No sales yet, so there is nothing to refund.
Receipt number	1 to the number of receipts	Please enter a whole number from 1 to N.
Refund twice	Only once per receipt	Receipt #N was already refunded.
Any question	Typing cancel	Cancelled. Nothing changed.

8.1 Ask-again questions

BookstoreUI.java lines 79–103

```
79  // Keeps asking until something is typed. Typing "cancel" stops the current task.
80  public String askText(String question) {
81      while (true) {
82          // Draw a purple bar, then \r jumps back so the typing starts right after the question.
83          System.out.print(PURPLE + String.format(" %-78s", question + " >") + "\r " + question +
    " > ");
84          if (!in.hasNextLine()) System.exit(0); // input was closed
85          String answer = in.nextLine().trim();
86          System.out.print(BODY);
87          if (answer.equalsIgnoreCase("cancel")) throw new RuntimeException("Cancelled. Nothing
    changed.");
88          if (!answer.isEmpty()) return answer;
89      }
90  }
…
92  // Asks again until the answer is a whole number from min to max.
93  public int askNumber(String question, int min, int max) {
94      while (true) {
95          try {
96              int number = Integer.parseInt(askText(question));
97              if (number >= min && number <= max) return number;
98          } catch (NumberFormatException e) {
99              // not a number, so ask again
100         }
101         error("Please enter a whole number from " + min + " to " + max + ".");
102     }
103 }
```

`askText` draws the purple input bar, reads a line and returns it (empty answers are ignored). If the user types `cancel` it throws an exception that ends the task. `askNumber` and `askMoney` keep asking until the answer is a number in the allowed range; a `NumberFormatException` (e.g. “abc”) is caught and simply leads to another try. If the input stream is closed, the program ends cleanly.

8.2 Rules inside Bookstore

Bookstore.java lines 61–91

```
61  // Members get 5% off. The total is rounded to 2 decimal places.
62  public double total(Product p, int quantity, boolean member) {
63      if (quantity < 1 || quantity > p.getStock())
64          throw new RuntimeException("Quantity must be from 1 to " + p.getStock() + ".");
65      double total = p.getPrice() * quantity;
66      if (member) total = total - p.calculateDiscount(5) * quantity;
67      return Math.round(total * 100) / 100.0;
68  }
…
70  public Sale sell(Product p, int quantity, boolean member, double paid) {
71      double total = total(p, quantity, member);
72      if (paid < total) throw new RuntimeException("Cash is less than the total. Sale
    cancelled.");
73      p.setStock(p.getStock() - quantity);
74      Sale sale = new Sale(sales.size() + 1, p, quantity, total, paid);
75      sales.add(sale);
76      save();
77      return sale;
78  }
…
85  public void refund(Sale sale) {
86      if (sale.refunded) throw new RuntimeException("Receipt #" + sale.id + " was already
    refunded.");
87      sale.refunded = true;
88      Product p = get(sale.productId);
89      p.setStock(p.getStock() + sale.quantity);
90      save();
91  }
```

Each rule that fails throws a `RuntimeException` with a clear message. The menu loop in `Main` catches it and shows it in a red bar, then returns to the menu (see the `try … catch` in Section 5.4).

9 Data Storage

All products and sales are stored inside one `Bookstore` object. Because `Bookstore`, `Product`, `Book`, `Stationery` and `Sale` implement `Serializable`, Java can write the whole object — with every product and sale inside it — to a file in one step, and read it back in one step.

Bookstore.java lines 93–110

```
93  // Saves the whole bookstore (products and sales) to the file in one step.
94  private void save() {
95      try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(FILE))) {
96          out.writeObject(this);
97      } catch (IOException e) {
98          throw new RuntimeException("Could not save: " + e.getMessage());
99      }
100 }
 …
102 // Reads the saved bookstore, or starts an empty one the first time.
103 public static Bookstore load() {
104     if (!new File(FILE).exists()) return new Bookstore();
105     try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(FILE))) {
106         return (Bookstore) in.readObject();
107     } catch (Exception e) {
108         throw new RuntimeException("Could not read " + FILE + ": " + e.getMessage());
109     }
110 }
```

Situation	What happens
First start (no file)	An empty bookstore is created.
Every change (add, restock, price, sell, refund)	The whole bookstore is saved immediately, so nothing is lost if the window is closed.
Normal start	The saved bookstore is loaded, with every product and receipt.
Damaged file	A clear message is shown and the program stops without overwriting the file (exit code 1, so run.bat keeps the window open).
After editing the code	Every class has `serialVersionUID = 1L`, so files saved by an older build can still be read.
GitHub	`bookstore.dat` and compiled `.class` files are listed in `.gitignore` and never uploaded.

10 Business Rules

Rule	How it is calculated / enforced	Code
Member discount	5% of the unit price for each item: total = price × quantity − calculateDiscount(5) × quantity	Bookstore.total, Product.calculateDiscount
Rounding	Totals and amounts are rounded to 2 decimal places	Math.round(x * 100) / 100.0
Change	Change = cash received − total; cash must be ≥ total	Bookstore.sell
Receipt number	Receipts are numbered 1, 2, 3 … in order	sales.size() + 1
Receipt keeps old data	A sale stores the title and total at the time of sale; later price changes do not change old receipts	class Sale
Refund	Whole receipt only, once; units go back to stock; refunded income is not counted	Bookstore.refund, showReport
Low stock	3 units or fewer: yellow row with “!” and counted in the report	showProducts, showReport
Limits	Stock 0–1,000,000 per entry; price up to 1,000,000,000 BDT	askNumber, askMoney

Product.java lines 71–73

```
71 public double calculateDiscount(double percentage) {
72     return price * percentage / 100;
73 }
```

11 Testing

The program was tested in three ways, always on copies of the data so real data was never touched:

- **Automated test cases** — 30 scripted sessions run the compiled program, type the inputs and check the exact text on the screen, the exit code and that no Java exception appeared. Result: **30 of 30 passed**.
- **Random input testing** — 50 sessions typed random menu choices and random answers (letters, huge numbers, negative numbers, “NaN”, empty lines, very long text, cancel …). After each session the data file was reloaded to make sure it was still valid. Result: **0 crashes, 0 broken data files**.
- **Real window testing** — the program was opened with `run.bat` in an 80 × 24 Windows Terminal window and driven key by key; all screenshots in this report come from those runs.

ID	Area	Test	Input	Expected result	Result
TC01	Add	Add a book with valid data	`3 / B / B2 / Head First Java / 950 / 8 / Kathy Sierra / 978-0596009205`	Added BOOK "Head First Java" by Kathy Sierra.	PASS
TC02	Add	Add stationery with valid data	`3 / S / S2 / A4 Notebook / 120 / 3 / Matador`	Added STATIONERY "A4 Notebook" from Matador.	PASS
TC03	Add	Add a book without ISBN (type -)	`3 / B / B3 / Poems / 200 / 4 / Ann / - / 4 / B3`	Book by Ann, ISBN N/A	PASS
TC04	Validation	Wrong product type letter (X)	`3 / X`	Please type B or S.	PASS
TC05	Validation	Duplicate product ID	`3 / B / B1`	ID B1 is already used. Please choose another.	PASS
TC06	Validation	Price that is not a number (abc)	`3 / B / B4 / T / abc`	Please enter an amount like 250 or 99.50.	PASS
TC07	Validation	Negative price (-5)	`3 / B / B4 / T / -5`	Please enter an amount like 250 or 99.50.	PASS
TC08	Validation	Stock that is not a whole number (x)	`3 / B / B4 / T / 10 / x`	Please enter a whole number from 0 to 1000000.	PASS
TC09	Search	Search that matches	`2 / clean`	Products shown: 1	PASS
TC10	Search	Search with no match	`2 / zzz`	No products found.	PASS
TC11	Restock	Unknown product ID	`4 / NOPE`	No product has that ID.	PASS
TC12	Restock	Quantity 0 is refused	`4 / S1 / 0`	Please enter a whole number from 1 to 1000000.	PASS
TC13	Restock	Valid restock	`4 / S1 / 10`	Restocked "Gel Pen". Stock is now 30.	PASS
TC14	Price	Change price	`5 / B1 / 450.50`	Price of "Clean Code" is now 450.50 BDT.	PASS
TC15	Sell	Quantity more than stock	`6 / B1 / 9`	Please enter a whole number from 1 to 5.	PASS

ID	Area	Test	Input	Expected result	Result
TC16	Sell	Member discount 5% (2 x 500)	`6 / B1 / 2 / y`	Total: 950.00 BDT	PASS
TC17	Sell	Cash less than the total	`6 / B1 / 2 / y / 900`	Please enter at least 950.00 BDT.	PASS
TC18	Sell	Valid sale with change	`6 / B1 / 2 / y / 1000`	Sale saved. Give back 50.00 BDT change.	PASS
TC19	Sell	'yes' is accepted as yes	`6 / S1 / 1 / yes / 100`	Total: 14.25 BDT	PASS
TC20	Sell	Out-of-stock product	`6 / B1 / 5 / n / 2500 / 6 / B1`	"Clean Code" is out of stock. Choose another product.	PASS
TC21	Cancel	Typing cancel stops a task	`6 / B1 / cancel`	Cancelled. Nothing changed.	PASS
TC22	Refund	Refund when there are no sales	`7`	No sales yet, so there is nothing to refund.	PASS
TC23	Refund	Answer n keeps the sale	`6 / S1 / 1 / n / 15 / 7 / 1 / n`	Not refunded. Nothing changed.	PASS
TC24	Refund	Valid refund returns stock	`6 / S1 / 2 / n / 30 / 7 / 1 / y`	Refunded. Give back 30.00 BDT. Stock returned.	PASS
TC25	Refund	Same receipt cannot be refunded twice	`6 / S1 / 2 / n / 30 / 7 / 1 / y / 7 / 1`	Receipt #1 was already refunded.	PASS
TC26	Report	Income ignores refunded sales	`6 / S1 / 2 / n / 30 / 6 / B1 / 1 / n / 500 / 7 / 1 / y / 9`	Income after refunds (BDT) 500.00	PASS
TC27	Menu	Menu choice that does not exist	`x`	Please choose a number from the menu.	PASS
TC28	Storage	Data is still there after restarting	`sell, exit, start again, 8`	Receipt #1 is listed	PASS
TC29	Storage	Damaged data file is not overwritten	`bookstore.dat contains 'garbage'`	Error message, exit code 1, file unchanged	PASS
TC30	Storage	Input ends in the middle of Add product	`3 / B / HALF / (end of input)`	Program ends; half-typed product is not saved	PASS

Inputs are the answers typed after a small sample catalogue (a book B1 “Clean Code”, 500 BDT, stock 5, and a stationery item S1 “Gel Pen”, 15 BDT, stock 20) was added.

12 Limitations and Future Work

Current limitation	Possible improvement
One product per sale	A shopping cart with several products on one receipt
Products cannot be deleted or renamed	Edit and delete screens
Only one copy of the program should use the data file at a time	A database (for example SQLite) that supports several users
Windows console shows only English letters correctly (accented letters may be changed)	A graphical interface (JavaFX) with full Unicode support
No user accounts	Staff login with different permissions
Reports only on screen	Export of reports and receipts to PDF or Excel

13 Conclusion

The Bookstore Inventory Manager meets its objectives. It handles the daily work of a small shop — catalogue, stock, sales, refunds and reports — with clear screens, helpful messages and automatic saving. It is short (617 lines in four files) and uses simple Java, yet every OOP concept from the course appears inside a real feature: abstraction and inheritance in the product hierarchy, encapsulation in the protected fields, runtime polymorphism in the product table, overloading in the Book constructors and the restock methods, overriding in getType/getDetails/getCategory, static members in the currency and the session counter, `this` and `super` in the constructors, upcasting when a new product is added, and the interface diamond problem in the product category. All 30 test cases and 50 random-input sessions passed without a single crash.

A Appendix: Full Source Code

A.1 Product.java

Product.java 130 lines

```java
import java.io.Serializable;

// DIAMOND PROBLEM: Trackable and Promotable both extend Item, and both give their own
// default getCategory(). Product implements both, so Java forces Product to choose.
//
//                Item
//              /      \
//       Trackable    Promotable
//              \      /
//              Product
interface Item {
    default String getCategory() { return "Item"; }
}

interface Trackable extends Item {
    @Override
    default String getCategory() { return "Tracked stock"; }
}

interface Promotable extends Item {
    @Override
    default String getCategory() { return "Promotable"; }
}

// ABSTRACTION: Product is abstract, so the program can only make a Book or a Stationery.
// ENCAPSULATION: the fields are private and are used only through getters and setters.
public abstract class Product implements Trackable, Promotable, Serializable {
    private static final long serialVersionUID = 1L; // keeps saved files readable after code edits

    // STATIC variables belong to the class and are shared by every product.
    public static final String CURRENCY;
    private static int createdCount = 0; // products created while the program runs (shown in the Report)

    // STATIC BLOCK: runs once, when Java first loads the Product class.
    static {
        CURRENCY = "BDT";
    }

    // NON-STATIC (instance) variables: every product object has its own copy.
    private String id, title;
    private double price;
    private int stock;

    public Product(String id, String title, double price, int stock) {
        this.id = id; // THIS: this.id is the object's field, id is the parameter
        this.title = title;
        this.price = price;
```

```
48          this.stock = stock;
49          createdCount++;
50      }
51
52      // STATIC METHOD: called on the class itself, e.g. Product.getCreatedCount()
53      public static int getCreatedCount() { return createdCount; }
54
55      // NON-STATIC METHODS: getters and setters work on one object's own data.
56      public String getId() { return id; }
57      public String getTitle() { return title; }
58      public double getPrice() { return price; }
59      public int getStock() { return stock; }
60
61      // The setters protect the data: a negative value is refused.
62      public void setPrice(double price) {
63          if (price >= 0) this.price = price;
64      }
65
66      public void setStock(int stock) {
67          if (stock >= 0) this.stock = stock;
68      }
69
70      // Discount on one item, by percentage (used for the 5% member discount).
71      public double calculateDiscount(double percentage) {
72          return price * percentage / 100;
73      }
74
75      // DIAMOND PROBLEM SOLVED: we say exactly which parent versions to use.
76      @Override
77      public String getCategory() {
78          return Trackable.super.getCategory() + " & " + Promotable.super.getCategory();
79      }
80
81      // ABSTRACT METHODS: every subclass must write its own version (overriding).
82      public abstract String getType();
83      public abstract String getDetails();
84  }
85
86  // INHERITANCE: Book is a subclass of the superclass Product.
87  class Book extends Product {
88      private static final long serialVersionUID = 1L;
89      private String author, isbn;
90
91      // CONSTRUCTOR OVERLOADING 1: a book with an ISBN.
92      public Book(String id, String title, double price, int stock, String author, String isbn) {
93          super(id, title, price, stock); // SUPER: runs the Product constructor first
94          this.author = author;
95          this.isbn = isbn;
96      }
97
98      // CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor
    above.
99      public Book(String id, String title, double price, int stock, String author) {
100         this(id, title, price, stock, author, "N/A");
101     }
102
103     public String getAuthor() { return author; } // only a Book has an author
```

```
104
105     // METHOD OVERRIDING: Book's own versions of Product's abstract methods.
106     @Override
107     public String getType() { return "BOOK"; }
108
109     @Override
110     public String getDetails() { return "Book by " + author + ", ISBN " + isbn; }
111 }
112
113 // INHERITANCE: Stationery is another subclass of Product.
114 class Stationery extends Product {
115     private static final long serialVersionUID = 1L;
116     private String brand;
117
118     public Stationery(String id, String title, double price, int stock, String brand) {
119         super(id, title, price, stock); // SUPER: runs the Product constructor first
120         this.brand = brand;
121     }
122
123     public String getBrand() { return brand; } // only Stationery has a brand
124
125     @Override
126     public String getType() { return "STATIONERY"; }
127
128     @Override
129     public String getDetails() { return "Stationery by " + brand; }
130 }
```

A.2 Bookstore.java

Bookstore.java 130 lines

```java
import java.io.*;
import java.time.LocalDate;
import java.util.ArrayList;

// The bookstore rules and the data file. This class never prints anything.
public class Bookstore implements Serializable {
    private static final long serialVersionUID = 1L;
    private static final String FILE = "bookstore.dat";
    private ArrayList<Product> products = new ArrayList<>();
    private ArrayList<Sale> sales = new ArrayList<>();

    public ArrayList<Product> getProducts() { return products; }
    public ArrayList<Sale> getSales() { return sales; }

    // Returns the product with this ID, or null if there is none.
    public Product find(String id) {
        for (Product p : products) {
            if (p.getId().equalsIgnoreCase(id)) return p;
        }
        return null;
    }

    // Same as find, but stops with an error message if the ID does not exist.
    public Product get(String id) {
        Product p = find(id);
        if (p == null) throw new RuntimeException("No product with ID " + id + ".");
        return p;
    }

    public ArrayList<Product> search(String text) {
        ArrayList<Product> found = new ArrayList<>();
        for (Product p : products) {
            String all = p.getId() + " " + p.getTitle() + " " + p.getDetails();
            if (all.toLowerCase().contains(text.toLowerCase())) found.add(p);
        }
        return found;
    }

    public void add(Product p) {
        if (find(p.getId()) != null) throw new RuntimeException("ID " + p.getId() + " is
already used.");
        products.add(p);
        save();
    }

    // METHOD OVERLOADING: restock by product ID, or by the product object itself.
    public void restock(String id, int quantity) {
        restock(get(id), quantity);
    }

    public void restock(Product p, int quantity) {
        if (quantity < 1) throw new RuntimeException("Quantity must be at least 1.");
        p.setStock(p.getStock() + quantity);
```

```
        save();
    }

    public void changePrice(Product p, double price) {
        p.setPrice(price);
        save();
    }

    // Members get 5% off. The total is rounded to 2 decimal places.
    public double total(Product p, int quantity, boolean member) {
        if (quantity < 1 || quantity > p.getStock())
            throw new RuntimeException("Quantity must be from 1 to " + p.getStock() + ".");
        double total = p.getPrice() * quantity;
        if (member) total = total - p.calculateDiscount(5) * quantity;
        return Math.round(total * 100) / 100.0;
    }

    public Sale sell(Product p, int quantity, boolean member, double paid) {
        double total = total(p, quantity, member);
        if (paid < total) throw new RuntimeException("Cash is less than the total. Sale
cancelled.");
        p.setStock(p.getStock() - quantity);
        Sale sale = new Sale(sales.size() + 1, p, quantity, total, paid);
        sales.add(sale);
        save();
        return sale;
    }

    public Sale getSale(int number) {
        if (number < 1 || number > sales.size()) throw new RuntimeException("No receipt #" +
number + ".");
        return sales.get(number - 1);
    }

    public void refund(Sale sale) {
        if (sale.refunded) throw new RuntimeException("Receipt #" + sale.id + " was already
refunded.");
        sale.refunded = true;
        Product p = get(sale.productId);
        p.setStock(p.getStock() + sale.quantity);
        save();
    }

    // Saves the whole bookstore (products and sales) to the file in one step.
    private void save() {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(FILE))) {
            out.writeObject(this);
        } catch (IOException e) {
            throw new RuntimeException("Could not save: " + e.getMessage());
        }
    }

    // Reads the saved bookstore, or starts an empty one the first time.
    public static Bookstore load() {
        if (!new File(FILE).exists()) return new Bookstore();
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(FILE))) {
            return (Bookstore) in.readObject();
```

```
        } catch (Exception e) {
            throw new RuntimeException("Could not read " + FILE + ": " + e.getMessage());
        }
    }
}

// One sale (receipt). It keeps the title and total even if the product changes later.
class Sale implements Serializable {
    private static final long serialVersionUID = 1L;
    int id, quantity;
    String date, productId, title;
    double total, paid;
    boolean refunded;

    Sale(int id, Product p, int quantity, double total, double paid) {
        this.id = id;
        this.date = LocalDate.now().toString();
        this.productId = p.getId();
        this.title = p.getTitle();
        this.quantity = quantity;
        this.total = total;
        this.paid = paid;
    }
}
```

A.3 BookstoreUI.java

BookstoreUI.java 216 lines

```java
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Scanner;

// Everything the user sees or types goes through this class.
// Texts like "\033[...m" are ANSI color codes that the terminal shows as colors:
// "38;2;R;G;B" sets the text color and "48;2;R;G;B" sets the background color.
public class BookstoreUI {
    private static final String BODY = "\033[38;2;229;237;255;48;2;12;22;66m";     // light text on navy
    private static final String DIM = "\033[38;2;140;189;225;48;2;12;22;66m";      // soft blue on navy
    private static final String YELLOW = "\033[38;2;248;212;123;48;2;12;22;66m";   // yellow on navy
    private static final String CYAN = "\033[38;2;9;25;47;48;2;103;199;211m";      // dark on cyan
    private static final String PURPLE = "\033[38;2;255;241;255;48;2;97;43;123m";  // white on purple
    private static final String WHITE = "\033[38;2;12;22;66;48;2;222;233;255m";    // navy on white
    private static final String GREEN = "\033[38;2;13;43;31;48;2;132;210;164m";    // dark on green
    private static final String RED = "\033[38;2;255;239;239;48;2;137;35;50m";     // white on red
    private static final String RESET = "\033[0m";                                 // terminal's own colors
    private static final String[] MENU = {"Exit", "Products", "Search", "Add product", "Restock",
            "Change price", "Sell", "Refund", "Sales", "Report"};
    private Scanner in = new Scanner(System.in);

    public BookstoreUI() {
        System.out.print("\033]0;Bookstore\007"); // window title
        // Give the terminal its normal colors back when the program ends (even after Ctrl+C).
        Runtime.getRuntime().addShutdownHook(new Thread(() -> System.out.print(RESET)));
    }

    // Prints one line: the text in its color, then the rest of the line in navy.
    private void line(String color, String text) {
        System.out.println(color + text + BODY + "\033[K");
    }

    // Shortens text that is too long for a table column, e.g. "A Very Long Ti..."
    private String fit(String text, int width) {
        if (text.length() <= width) return text;
        return text.substring(0, width - 3) + "...";
    }

    // Clears the screen (and old screens above it), then draws the title bar, menu tabs and screen name.
    public void newScreen(int tab) {
        System.out.print(BODY + "\033[2J\033[3J\033[H");
        line(CYAN, String.format(" %-60s%17s ", "BOOKSTORE / INVENTORY MANAGER", "JAVA / LOCAL"));
```

```
        String tabs = "";
        for (int n = 1; n <= 10; n++) {
            int number = n % 10; // 1, 2, ... 9, then 0 for Exit
            String color = (number == tab) ? WHITE : BODY;
            tabs = tabs + color + String.format(" %d %-12s", number, MENU[number]) + BODY;
            if (n == 5 || n == 10) {
                line(BODY, tabs);
                tabs = "";
            }
        }
        String time = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm"));
        line(PURPLE, String.format(" %-60s%17s ", MENU[tab].toUpperCase(), time));
        line(BODY, "");
    }

    // The bottom of every screen: a hint, then the menu question.
    public String askMenuChoice() {
        line(DIM, " " + "-".repeat(77));
        line(DIM, " Type a menu number and press Enter. In any task, type cancel to stop.");
        return askText("Choose");
    }

    public void message(String text) {
        line(GREEN, String.format(" %-78s", text));
    }

    public void error(String text) {
        line(RED, String.format(" %-78s", text));
    }

    public void goodbye() {
        System.out.print(RESET + "\033[2J\033[H");
        System.out.println("Goodbye! Everything is saved.");
    }

    // Keeps asking until something is typed. Typing "cancel" stops the current task.
    public String askText(String question) {
        while (true) {
            // Draw a purple bar, then \r jumps back so the typing starts right after the
question.
            System.out.print(PURPLE + String.format(" %-78s", question + " >") + "\r " +
question + " > ");
            if (!in.hasNextLine()) System.exit(0); // input was closed
            String answer = in.nextLine().trim();
            System.out.print(BODY);
            if (answer.equalsIgnoreCase("cancel")) throw new RuntimeException("Cancelled.
Nothing changed.");
            if (!answer.isEmpty()) return answer;
        }
    }

    // Asks again until the answer is a whole number from min to max.
    public int askNumber(String question, int min, int max) {
        while (true) {
            try {
                int number = Integer.parseInt(askText(question));
```

```
97                 if (number >= min && number <= max) return number;
98             } catch (NumberFormatException e) {
99                 // not a number, so ask again
100            }
101            error("Please enter a whole number from " + min + " to " + max + ".");
102        }
103    }
104
105    // Asks again until the answer is an amount of at least min (rounded to 2 decimals).
106    public double askMoney(String question, double min) {
107        while (true) {
108            try {
109                double amount = Math.round(Double.parseDouble(askText(question)) * 100) /
    100.0;
110                if (amount >= min && amount <= 1000000000) return amount;
111            } catch (NumberFormatException e) {
112                // not a number, so ask again
113            }
114            error(min > 0 ? String.format("Please enter at least %.2f BDT.", min) : "Please
    enter an amount like 250 or 99.50.");
115        }
116    }
117
118    // Asks again until the answer starts with one of two letters, e.g. "YN" or "BS".
119    public char askLetter(String question, String letters) {
120        while (true) {
121            char letter = Character.toUpperCase(askText(question).charAt(0));
122            if (letters.indexOf(letter) >= 0) return letter;
123            error("Please type " + letters.charAt(0) + " or " + letters.charAt(1) + ".");
124        }
125    }
126
127    public boolean askYes(String question) {
128        return askLetter(question + " (y/n)", "YN") == 'Y';
129    }
130
131    // A table of products. Low stock (3 or fewer) is yellow and marked with "!".
132    public void showProducts(ArrayList<Product> list) {
133        line(CYAN, String.format(" %-9s %-21s %-29s %9s %6s", "ID", "TITLE", "DETAILS", "PRICE
    BDT", "STOCK"));
134        int low = 0;
135        for (Product p : list) {
136            String color = BODY;
137            String stock = p.getStock() + " ";
138            if (p.getStock() <= 3) {
139                color = YELLOW;
140                stock = p.getStock() + "!";
141                low++;
142            }
143            line(color, String.format(" %-9s %-21s %-29s %9.2f %6s",
144                    fit(p.getId(), 9), fit(p.getTitle(), 21), fit(p.getDetails(), 29),
    p.getPrice(), stock));
145        }
146        if (list.isEmpty()) line(DIM, " No products found. Choose 3 to add one.");
147        line(DIM, " Products shown: " + list.size() + "  |  Low on stock: " + low + " (yellow,
    marked !)");
148        line(BODY, "");
```

```
149     }
150
151     // One white "card" line, used for the product card and the receipt.
152     private void card(String text) {
153         line(WHITE, String.format("   %-44s ", fit(text, 44)));
154     }
155
156     public void showProduct(Product p) {
157         card(p.getType() + ": " + p.getTitle() + "  (" + p.getId() + ")");
158         card(p.getDetails());
159         card("Category: " + p.getCategory());
160         card(String.format("Price %.2f %s   |   In stock: %d", p.getPrice(), Product.CURRENCY,
    p.getStock()));
161         line(BODY, "");
162     }
163
164     public void showReceipt(Sale s) {
165         card("");
166         card("RECEIPT #" + s.id + "     " + s.date + "     " + (s.refunded ? "REFUNDED" :
    "PAID"));
167         card("- - - - - - - - - - - - - - - - - - - - - -");
168         card(s.title + "  x " + s.quantity);
169         card(String.format("Total            %14.2f BDT", s.total));
170         card(String.format("Cash received    %14.2f BDT", s.paid));
171         card(String.format("Change           %14.2f BDT", s.paid - s.total));
172         card("");
173         line(BODY, "");
174     }
175
176     // A table of all sales. Refunded sales are yellow.
177     public void showSales(ArrayList<Sale> list) {
178         line(CYAN, String.format(" %-5s %-10s %-24s %4s %10s %10s %-9s", "NO.", "DATE",
    "PRODUCT", "QTY", "TOTAL", "CHANGE", "STATUS"));
179         for (Sale s : list) {
180             String color = s.refunded ? YELLOW : BODY;
181             String status = s.refunded ? "REFUNDED" : "PAID";
182             line(color, String.format(" #%-4d %-10s %-24s %4d %10.2f %10.2f %-9s",
183                     s.id, s.date, fit(s.title, 24), s.quantity, s.total, s.paid - s.total,
    status));
184         }
185         if (list.isEmpty()) line(DIM, " No sales yet. Choose 6 to sell something.");
186         line(DIM, " Receipts: " + list.size() + "  |  Refunded receipts are yellow.");
187         line(BODY, "");
188     }
189
190     public void showReport(Bookstore store) {
191         double income = 0, stockValue = 0;
192         int unitsSold = 0, unitsInStock = 0, lowStock = 0;
193         for (Sale s : store.getSales()) {
194             if (!s.refunded) {
195                 income = income + s.total;
196                 unitsSold = unitsSold + s.quantity;
197             }
198         }
199         for (Product p : store.getProducts()) {
200             unitsInStock = unitsInStock + p.getStock();
201             stockValue = stockValue + p.getPrice() * p.getStock();
```

```
            if (p.getStock() <= 3) lowStock++;
        }
        line(CYAN, String.format(" %-78s", "SALES"));
        line(WHITE, String.format(" %-58s%19.2f ", "Income after refunds (BDT)", income));
        line(BODY, String.format(" %-58s%19d", "Units sold", unitsSold));
        line(BODY, "");
        line(CYAN, String.format(" %-78s", "STOCK"));
        line(BODY, String.format(" %-58s%19d", "Products", store.getProducts().size()));
        line(BODY, String.format(" %-58s%19d", "Low on stock (3 or fewer)", lowStock));
        line(BODY, String.format(" %-58s%19d", "Units in stock", unitsInStock));
        line(BODY, String.format(" %-58s%19d", "New products added this session",
Product.getCreatedCount()));
        line(WHITE, String.format(" %-58s%19.2f ", "Stock value at selling price (BDT)",
stockValue));
        line(BODY, "");
    }
}
```

A.4 Main.java

Main.java 141 lines

```java
import java.util.Locale;

// Starts the program and runs the menu option the user picks.
public class Main {
    static BookstoreUI ui = new BookstoreUI();
    static Bookstore store;

    public static void main(String[] args) {
        Locale.setDefault(Locale.US); // always show prices as 450.50 (with a dot) on every computer
        try {
            store = Bookstore.load();
        } catch (RuntimeException e) {
            ui.error(e.getMessage());
            System.exit(1); // stop with an error code, so run.bat keeps the window open to read this
        }
        ui.newScreen(1);
        ui.showProducts(store.getProducts());
        while (true) {
            try {
                String choice = ui.askMenuChoice();
                switch (choice) {
                    case "1": ui.newScreen(1); ui.showProducts(store.getProducts()); break;
                    case "2": ui.newScreen(2); search(); break;
                    case "3": ui.newScreen(3); addProduct(); break;
                    case "4": ui.newScreen(4); restock(); break;
                    case "5": ui.newScreen(5); changePrice(); break;
                    case "6": ui.newScreen(6); sell(); break;
                    case "7": ui.newScreen(7); refund(); break;
                    case "8": ui.newScreen(8); ui.showSales(store.getSales()); break;
                    case "9": ui.newScreen(9); ui.showReport(store); break;
                    case "0": ui.goodbye(); return;
                    default: ui.error("Please choose a number from the menu.");
                }
            } catch (RuntimeException e) {
                ui.error(e.getMessage()); // show the problem in red, then go back to the menu
            }
        }
    }

    // Asks for a product ID until it matches a product (or the user types cancel).
    static Product askProduct() {
        while (true) {
            Product p = store.find(ui.askText("Product ID"));
            if (p != null) return p;
            ui.error("No product has that ID. Check the table above and try again.");
        }
    }

    // After a task: draw the screen again with the updated table and a green message.
    static void showResult(int tab, String text) {
        ui.newScreen(tab);
```

```
        ui.showProducts(store.getProducts());
        ui.message(text);
    }

    static void search() {
        String text = ui.askText("Search for (ID, title, author, brand or ISBN)");
        ui.newScreen(2);
        ui.showProducts(store.search(text));
        ui.message("Results for \"" + text + "\"");
    }

    static void addProduct() {
        char type = ui.askLetter("Type: B = book, S = stationery", "BS");
        String id = ui.askText("ID (short code, e.g. B1)");
        while (store.find(id) != null) {
            ui.error("ID " + id + " is already used. Please choose another.");
            id = ui.askText("ID (short code, e.g. B1)");
        }
        String title = ui.askText("Title");
        double price = ui.askMoney("Price (BDT)", 0);
        int stock = ui.askNumber("Stock", 0, 1000000);
        Product product; // SUPERCLASS reference: can hold a Book or a Stationery
        String extra;
        if (type == 'B') {
            String author = ui.askText("Author");
            String isbn = ui.askText("ISBN (type - if it has none)");
            Book book; // SUBCLASS reference
            if (isbn.equals("-")) book = new Book(id, title, price, stock, author); //
overloaded constructor
            else book = new Book(id, title, price, stock, author, isbn);
            extra = " by " + book.getAuthor(); // getAuthor() works only through the Book
reference
            product = book;                    // UPCASTING: the Book is stored as a Product
        } else {
            Stationery item = new Stationery(id, title, price, stock, ui.askText("Brand"));
            extra = " from " + item.getBrand();
            product = item;
        }
        store.add(product);
        showResult(3, "Added " + product.getType() + " \"" + title + "\"" + extra + ".");
    }

    static void restock() {
        ui.showProducts(store.getProducts());
        Product p = askProduct();
        ui.showProduct(p);
        store.restock(p.getId(), ui.askNumber("Quantity to add", 1, 1000000)); // overloaded
restock(String, int)
        showResult(4, "Restocked \"" + p.getTitle() + "\". Stock is now " + p.getStock() +
".");
    }

    static void changePrice() {
        ui.showProducts(store.getProducts());
        Product p = askProduct();
        ui.showProduct(p);
        store.changePrice(p, ui.askMoney("New price (BDT)", 0));
```

```
105         showResult(5, String.format("Price of \"%s\" is now %.2f BDT.", p.getTitle(),
    p.getPrice()));
106     }
107
108     static void sell() {
109         ui.showProducts(store.getProducts());
110         Product p = askProduct();
111         while (p.getStock() == 0) {
112             ui.error("\"" + p.getTitle() + "\" is out of stock. Choose another product.");
113             p = askProduct();
114         }
115         ui.showProduct(p);
116         int quantity = ui.askNumber("Quantity (1-" + p.getStock() + ")", 1, p.getStock());
117         boolean member = ui.askYes("Member? 5% off");
118         double total = store.total(p, quantity, member);
119         ui.message(String.format("Total: %.2f BDT", total));
120         Sale sale = store.sell(p, quantity, member, ui.askMoney("Cash received (BDT)", total));
121         ui.newScreen(6);
122         ui.showReceipt(sale);
123         ui.message(String.format("Sale saved. Give back %.2f BDT change.", sale.paid -
    sale.total));
124     }
125
126     static void refund() {
127         if (store.getSales().isEmpty()) throw new RuntimeException("No sales yet, so there is
    nothing to refund.");
128         ui.showSales(store.getSales());
129         Sale sale = store.getSale(ui.askNumber("Receipt number", 1, store.getSales().size()));
130         if (sale.refunded) throw new RuntimeException("Receipt #" + sale.id + " was already
    refunded.");
131         ui.showReceipt(sale);
132         if (!ui.askYes("Refund this sale")) {
133             ui.message("Not refunded. Nothing changed.");
134             return;
135         }
136         store.refund(sale);
137         ui.newScreen(7);
138         ui.showReceipt(sale);
139         ui.message(String.format("Refunded. Give back %.2f BDT. Stock returned.", sale.total));
140     }
141 }
```

A.5 run.bat

run.bat 10 lines

```
@echo off
rem Compiles the program, then opens it in its own window that is exactly its size: 80 x 24 characters.
cd /d "%~dp0"
javac *.java || (pause & exit /b 1)
where wt >nul 2>nul
if errorlevel 1 (
    start "Bookstore" cmd /c "mode con cols=80 lines=24 & java Main || pause"
) else (
    start "" wt -w new --size 80,24 --pos 0,0 -d . cmd /c "java Main || pause"
)
```